Formalizing the User-Kernel Shared-Memory Boundary: The C/LKMM Compound Model

Anonymous Author(s)

Abstract

The boundary between an operating-system kernel and its user-space applications has become a load-bearing surface: zero-copy interfaces such as `io_uring` and BPF arenas now place shared memory directly between code that obeys the C memory model and code that obeys the Linux Kernel Memory Model (LKMM). No formal model spans this boundary, and the two underlying models are structurally different; C builds happens-before from explicit release-acquire pairs, while the LKMM builds it from preserved program order.

We present the C/LKMM compound memory model, the first formal memory model specification of the user-kernel boundary. The compound model takes the LKMM as its structural basis and bridges it with C through two ideas: a *synchronize-threads* relation that parameterizes synchronization strength by the language of the operations involved, and *fence context sharing*, which lets a single LKMM fence context participate in multiple synchronizations without leaving its native context. We prove three correctness properties: pure-C programs retain their C semantics, pure-Linux programs retain their LKMM semantics, and shared synchronization primitives are interchangeable across the boundary—and complement them with systematic litmus-test validation. Using the model as a guide, we develop a family of lock-free data structures shared between user-space C and eBPF kernel code over BPF arenas. Finally, we identify and fix a real cross-boundary memory ordering bug in the Linux kernel's `kcov` subsystem; both halves of the fix have been accepted upstream.

1 Introduction

The boundary between an operating system kernel and user-space applictions is undergoing a fundamental shift. To meet the demands of modern hardware, user-space programs and the kernel increasingly share data directly, synchronizing in a zero-copy fashion using interfaces like `io_uring` and eBPF. This intimacy raises a key question: what defines correctness in these user-kernel interactions? A formal, unified memory model that describes this boundary is conspicuously absent.

Correctness for user-space is defined by the C memory model [11, 13], while the kernel adheres to the Linux Kernel Memory Model (LKMM) [2, 15]. Each has evolved to serve its domain, resulting in two designs with different approaches to ordering. In C, cross-thread ordering is governed almost exclusively by release-acquire pairs; it only exists where the programmer explicitly asks for it. The LKMM, in contrast, relies on *preserved program order* (ppo): a designated set of intra-thread patterns—address dependencies, data dependencies, and others—that the hardware and compiler are required to respect. Cross-thread ordering then arises whenever one thread reads a value written by another, linking the writer's ppo to the reader's ppo; no explicit release or acquire is required. The address-dependency case is a key example: in the LKMM, a pointer read followed by its dereference is automatically ordered, and this is a standard kernel synchronization pattern; in C, the same code orders nothing unless the pointer read is tagged `acquire`. A similar difference shows up with fences. C treats fences as independent synchronization points, while the LKMM defines each fence only in context with the surrounding memory operations it orders, a design that reflects how fences compile to hardware.

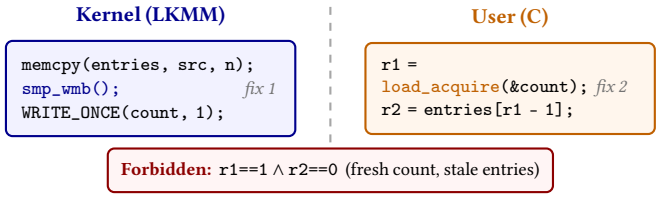

Figure 1. The `kcov_remote` cross-boundary bug. `load_acquire` abbreviates `__atomic_load_n(..., __ATOMIC_ACQUIRE)`. Neither fix alone fixes the bug: `smp_wmb()` on the kernel side has no effect if the user-side load is C-relaxed, and an acquire load on the user side has no effect if the kernel's stores are unordered under LKMM. The bug is closed only by a matched pair across the boundary—precisely the contract of our C/LKMM compound memory model.

Each model serves its domain well in isolation, but the boundary between them is now a load-bearing surface, and intuitions carried over from one domain can lead to subtle, hard-to-diagnose concurrency bugs in code where user-space and kernel interact.

A concrete example illustrates the problem (Figure 1). The Linux kernel's `kcov_remote` shares a buffer with a user-space consumer: the first word holds a count of valid coverage entries, and the rest hold the entries themselves. The kernel appends new entries by `memcpy`-ing them into the buffer and then updating the count; user space loads the count and then reads that many entries. Until recently, however, the kernel-side producer omitted a write barrier between the `memcpy` and the count update, and the user-side consumer loaded the count with a C relaxed atomic. Each omission, in isolation, is diagnosable against its own model—the kernel code is unordered under LKMM without an explicit barrier, and C enforces no ordering for reads after a relaxed load, nor does it honor the

address dependency from the count to the entry read. What is striking is that both omissions happened at the kernel-user interface. We read this as a symptom of the absence of a contract across the boundary: with no formal statement of the cross-boundary contract, it is natural for a programmer on either side to assume—incorrectly—that the other side, or the interface itself, supplies the missing ordering. The fix bears this out: closing the bug requires a matched pair, `smp_wmb()` on the producer and an acquire load on the consumer, and neither alone suffices—the kernel barrier orders its own stores but cannot force a relaxed C load to observe them in order, and an acquire load on the user side has nothing to synchronize against if the kernel's stores remain unordered. We identified the bug, proposed both fixes, and both patches have been accepted upstream. The matched release/acquire pair across the boundary is precisely the contract our compound memory model expresses.

We argue that this boundary must be placed on a solid formal foundation. This foundation must reflect the intuition that the core synchronization primitives of each domain should be semantically *interchangeable*, allowing a C release to correctly synchronize with a kernel acquire, and vice versa. To this end, we present the C/LKMM compound memory model, the first formal specification of the user-kernel memory boundary. Our compound model preserves the exact semantics of pure C and pure Linux programs—neither model is modified in isolation—and unifies them by extending both the happens-before and the sequentially-consistent axioms across the boundary, so that releases, acquires, and SC fences remain semantically interchangeable between the two languages.

The compound model takes the LKMM as its structural basis and extends it with two ideas that together deliver interchangeability. First, we introduce a *synchronize-threads* relation (`st`) that parametrizes synchronization strength by the language of the operations involved: a cross-thread synchronization involving C operations is held to C's release-acquire requirement, while one between LKMM operations follows LKMM's native ppo rules. Second, we introduce *fence context sharing*: we treat each LKMM fence together with its required surrounding operations as a single inseparable unit, and allow the same unit to participate in multiple synchronizations at once—for example, as the acquire-side of one synchronization and the release-side of another—without leaving its native context. Our model covers all the fences and synchronization primitives of both C and LKMM, except C's SC atomics, which have no LKMM analogue and are not expected to feature in user-kernel synchronization.

We formally prove three correctness properties of the compound model: pure-C programs retain their C semantics, pure-Linux programs retain their LKMM semantics, and the shared synchronization primitives are interchangeable across the boundary. We complement these proofs with systematic litmus testing. For the preservation properties, we run thousands of existing C and LKMM litmus tests under both their native model and the compound model, confirming that the allowed behaviors match in each case. For interchangeability, we take the same tests and systematically substitute each shared synchronization primitive with its counterpart in the other language, confirming that the allowed behaviors remain unchanged.

To demonstrate the model's utility, we develop a family of lock-free concurrent data structures shared between user-space C code and eBPF kernel code. The BPF memory model is under active formalization [5, 16]; we treat eBPF kernel code as LKMM-governed, consistent with the proposed model on the synchronization operations our case studies use. Each data structure runs on BPF Arenas, which provide zero-syscall shared memory between user space and the kernel. We apply the compound model to annotate producer and consumer paths separately for C and LKMM, and reason about heterogeneous synchronization across the boundary. The resulting implementations are both worked examples of compound-model reasoning and standalone contributions to user-kernel concurrent programming.

Contributions.

- The *C/LKMM compound memory model*, the first formal specification of the user-kernel memory boundary.
- Formal proofs of three correctness properties—pure-C programs retain their C semantics, pure-Linux programs retain their LKMM semantics, and the shared synchronization primitives are interchangeable across the boundary—complemented by systematic litmus-test validation.
- A family of lock-free concurrent data structures shared between user-space C code and eBPF kernel code over BPF arenas—including the Michael-Scott queue, ring buffers, and stacks—that serve as worked case studies in compound-model reasoning and are of standalone interest for user-kernel concurrent programming.
- A real-world cross-boundary bug, identified using the compound model as a formal guide: a memory ordering bug in the Linux kernel's `kcov` subsystem that requires a matched pair of fixes—a write barrier on the kernel producer and an acquire load on the user consumer—both of which have been accepted upstream.

2 Background

The C [11, 13] and Linux Kernel Memory Model (LKMM) [2, 15] differ fundamentally in design philosophy; we give a brief primer on each below and refer readers to the cited works for full descriptions.

2.1 The C Memory Model

The C memory model builds cross-thread ordering from explicit synchronization primitives. The core relation is "synchronizes with" (sw): for instance, a release write to a location read by an acquire on another thread.

Figure 2 shows the standard message-passing pattern. The release-acquire pair creates a sw edge from the flag write to the flag read. Combined with sequenced-before (sb) on each thread, this gives a "happens-before" path from the data write to the data read. C enforces ordering by requiring acyclicity of hb composed with communication: if the data read were to see the stale value, a from-read (fr) edge would close a cycle, and the coherence axiom would forbid it.

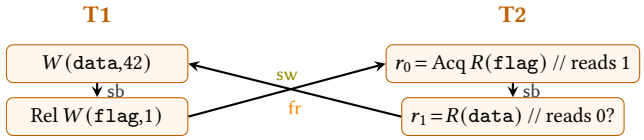

Figure 2. Message passing in C. The release-acquire pair creates an sw edge. If the data read sees the stale value, fr closes a cycle with hb, forbidden by C's coherence axiom.

Formally, C's happens-before relation is defined as

$$\mathsf{hb} = (\mathsf{sb} \mid \mathsf{sw})^+$$

where sb (sequenced-before) is program order and $^+$ denotes transitive closure. The coherence axiom requires

$$\text{acyclic } \mathsf{hb};\mathsf{eco}^?$$

where $\mathsf{eco} = (\mathsf{rf} \mid \mathsf{co} \mid \mathsf{fr})^+$ and $^?$ is the reflexive closure. The cycle in Figure 2—hb from the data write to the data read, then fr back—is exactly what this axiom forbids. A key point: co and fr never enter hb itself—C builds hb exclusively from program order and release-acquire synchronization, and uses eco only to check that this ordering is consistent with communication. LKMM, as we shall see, takes a different approach.

C's second axiom, PSC, governs SC fences and atomics. The full axiom requires acyclic (pscb | pscf), but since we omit SC accesses [13] (justified in Section 3), only pscf is relevant:

$$\mathsf{pscf} = [\mathsf{F} \cap \mathsf{sc}];(\mathsf{hb} \mid \mathsf{hb};\mathsf{eco};\mathsf{hb})\,[\mathsf{F} \cap \mathsf{sc}]$$

The endpoints of pscf are the fences themselves—a structural difference from the LKMM, where fences must remain embedded in the context of their surrounding operations.

2.2 The Linux Kernel Memory Model

In the LKMM, inter-thread synchronization is permissive: any cross-thread read-from (rfe) is sufficient—no release or acquire is needed. The burden shifts instead to intra-thread ordering. Where C treats all of program order as happens-before, the LKMM preserves only the subset that the hardware and compiler are required to respect: address dependencies, data dependencies, control dependencies, and fences. This subset is called *preserved program order* (ppo).[1]

[1]The LKMM assumes that address and control dependencies compile to real hardware ordering—an assumption our compound model inherits. Current compilers do not formally guarantee this, and there is ongoing work [9].

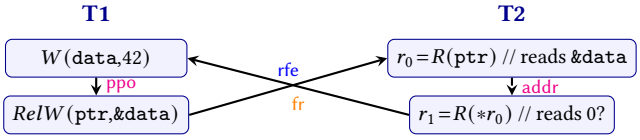

Figure 3. Address-dependency ordering in the LKMM. The address dependency from $R(\texttt{ptr})$ to $R(*r_0)$ is ppo; combined with the LKMM hb ordering from $R(*r_0)$ to $R(ptr)$, this forbids the stale read. In C, the same code provides no ordering; $R(\texttt{ptr})$ would need an acquire annotation.

Figure 3 illustrates this: the address dependency from a pointer read to its dereference is ppo, giving the LKMM an ordering guarantee that C does not provide without an explicit acquire.

The LKMM's happens-before has a third component beyond ppo and rfe: the *propagation* relation (prop). In that sense, the LKMM is closer to hardware models, which provide propagation guarantees via inter-thread co and fr edges in the presence of appropriate cumulative fences (cumul-fence). The prop relation chains an optional external co or fr edge with cumul-fence orderings to derive happens-before ordering—where C keeps co and fr outside hb entirely. The resulting definition is:

$$\mathsf{hb} = \mathsf{ppo} \mid \mathsf{rfe} \mid (\mathsf{prop} \setminus \mathsf{id}) \cap \mathsf{int}$$

Unlike C's happens-before, this is not a transitive closure—each component contributes directly, and the $\setminus id$ restriction prevents trivial self-ordering.

The LKMM's counterpart to pscf is *propagates-before* (pb), but its structure differs because the LKMM treats fences as meaningful only in the context of the operations they order. For example, a `Before_atomic` fence provides ordering guarantees only when followed by an atomic RMW; outside this context it has no meaning, which reflects how it compiles to hardware—on x86, `Before_atomic` compiles to a no-op since it would be redundant against the strong ordering guarantees of the following RMW. Due to this requirement, fences cannot be the endpoints of the relation as they are in C's pscf.

The LKMM defines the propagates-before relation $\mathsf{pb} = \mathsf{prop};\mathsf{strong\text{-}fence};\mathsf{hb}^*$ and requires acyclic pb, keeping the fence in the center of its context. This structural difference—fences as endpoints in C versus fences in context in the LKMM—is one of the core challenges the compound model must reconcile (Section 3).

2.3 Comparing C and LKMM

Table 1 highlights some of these key differences in design.

3 Overview

3.1 Challenges

Constructing a compound model is nontrivial due to the structural differences outlined above. Two problems arise.

	C	LKMM
Inter-Thread	Rel-Acq	Any reads-from (rfe)
Intra-Thread	Prog. Order (po)	Preserved (ppo)
Non-`rfe` Comm.	Outside *hb* (eco)	Inside *hb* (prop)
Fences	Standalone Sync	Requires Context
SC Axiom	Endpoints (pscf)	Central (pb)
Philosophy	Portable Abstr.	Hardware-Centric

Table 1. Summary of Differences between C and LKMM

Problem 1: Defining Synchronization. The two models cannot simply be unioned. Starting from C and adding LKMM's preserved program orders does not work: *ppo* is already a subset of program order, so C absorbs it with no effect, while C's requirement for release-acquire synchronization remains overly strict for Linux code. Starting from the LKMM is more promising, but raises its own questions: C depends on *all* program-order edges being preserved, which conflicts with the LKMM's restricted *ppo*, and the LKMM's permissive inter-thread synchronization—any *rfe* suffices—conflicts with C's requirement that cross-thread ordering be mediated by release-acquire pairs.

Problem 2: Matching the Fence Requirements of C and Linux. C and the LKMM treat fences in fundamentally different ways. In C, a fence is a standalone synchronization point: in Figure 4, the SC fence acts as an acquire to synchronize with T1's release, and as a release to synchronize with T3's acquire—participating in two *sw* edges simultaneously.

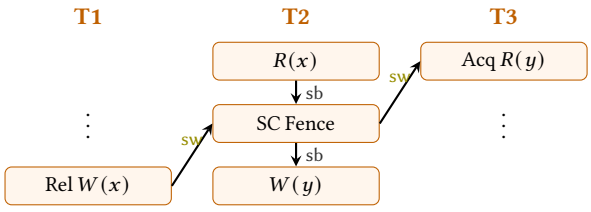

Figure 4. SC fence synchronization in C. The SC fence participates in two synchronizations: as an acquire (synchronizing with T1's release via sw) and as a release (synchronizing with T3's acquire via sw). In the LKMM, the fence has no standalone meaning—it is only meaningful in the context of the surrounding operations $R(x)$;SC Fence;$W(y)$.

In LKMM, a fence has no meaning on its own—it is a pattern-matching trigger whose semantics depend on the operations that surround it. A `Before_atomic` fence, for example, provides ordering only when followed by an atomic RMW; without this context, it compiles to a no-op on x86 (Section 2.2).

This difference creates a dilemma for the compound model. Following C's approach and treating fences as standalone endpoints would be unsound for LKMM fences. A `Before_atomic` not followed by an RMW would be credited with ordering it does not provide. Following the LKMM's approach—requiring fences to remain embedded in their surrounding context—breaks C's synchronization patterns. In Figure 4, $R(x)$ is relaxed and $W(y)$ is relaxed; only the SC fence provides acquire and release behavior. If the compound model does not allow fences to participate in synchronization, neither cross-thread edge can be established, and the model would allow T3 to read stale data—a behavior C forbids. A new approach is needed that respects the LKMM's context requirement while recovering C's fence synchronization patterns.

3.2 Approach

We take the LKMM as the structural basis for the compound model, with two modifications.

Synchronize-threads (st). We replace rfe in the LKMM's happens-before with an abstract relation, st, that parametrizes synchronization requirements by language. Between two LKMM operations, st requires only rfe—the LKMM default. When both operations are C, st requires a release on the writer and an acquire on the reader, matching C's sw requirements. In the cross-boundary case, each side contributes only what its own language demands: a kernel write need not be a release to synchronize with a C acquire, and a kernel read need not be an acquire to synchronize with a C release.

To recover C's intra-thread ordering, we extend ppo with *po-rel* and *acq-po*: any operation before a release, or after an acquire, is preserved. These are the implicit program orders that C's sw-based happens-before relies on. Additional cases for synchronization in C, such as intervening RMWs between release and acquire, are encoded in st without impacting Linux behavior.

Fence context sharing. To resolve Problem 2, we treat each fence together with its surrounding operations as a unit and allow the unit to participate in multiple synchronizations. In Figure 4, the release on T1 synchronizes not with the SC fence alone but with the entire context $R(x)$;SC Fence;$W(y)$, and this same context synchronizes with the acquire on T3. This preserves the LKMM's requirement that fences remain embedded in context—ensuring that pattern-dependent fences like `Before_atomic` are never credited with ordering outside their context—while recovering C's ability to route multiple synchronizations through a single fence.

4 A Compound Model

We take the LKMM as the structural basis and extend it in two steps: §4.1 introduces the main changes to synchronization and preserved program order; §4.2 develops the generalized relation composition needed to reconcile C and LKMM fence semantics; §4.3 gives the complete definition.

4.1 Major Changes From the LKMM

The main changes to the LKMM for building the compound memory model relate to the happens-before relation, which in LKMM is defined as:

$$\mathsf{hb} = \mathsf{ppo} \mid \mathsf{rfe} \mid (\mathsf{prop} \setminus \mathsf{id}) \cap \mathsf{int}$$

In the following, we discuss changes both to the relations defining it and the corresponding axiom.

Synchronization. The difficulty defining synchronization is that C and LKMM have different strength requirements for when they synchronize, and we need to be able to respect them simultaneously. In Linux, an rfe is enough, whereas in C it requires a release or an acquire, respectively. To cover both cases, we replace rfe with st (Eqn. 9) in hb, to define synchronization for both C and Linux.

As defined in LKMM, st relies on the notion of Marked: An operation is Marked if it uses an LKMM accessor (one of `READ_ONCE()`, `smp_store_release()`, etc.); C operations are Plain. A Marked operation satisfies the predicates that restrict the start and end of a synchronization trivially—no annotation is needed, recovering the LKMM's permissive rfe.

Extended preserved program order. C's happens-before includes all of sequenced-before (sb), but the compound model inherits the LKMM's restricted ppo, which preserves only specific intra-thread patterns. To recover the program orders that C relies on, we modify the po-rel and acq-po relations of the LKMM. The full definitions (Eqns. 5-6) use a generalized notion of composition, $\fatsemi$, which we explain in Subsection 4.2. Using this, we define two new notions, inter-start and inter-end (Eqn. 7 and Eqn. 8). C only satisfies inter-start if it uses a release (or a release fence via po-rel), and inter-end only if it uses an acquire (or an acquire fence via acq-po), recovering C's sw requirements. The strong-fence disjunct covers SC fences (`smp_mb()` and `memory_order_seq_cst` fences); weak-fence covers C's acquire-release fence. In the cross-boundary case, each side is checked independently: a kernel write need not be a release to synchronize with a C acquire, and vice versa.

The middle term $(\mathrm{rfe} \cup (\mathrm{rf} \fatsemi \mathrm{rmw})^+ \fatsemi \mathrm{rf}^?)$ allows intervening RMWs between release and acquire, matching C's treatment of release-acquire chains. Here, $\fatsemi$ refers to the generalized composition that we will introduce in Subsection 4.2. Notably, while this definition is more permissive than using rfe, it does not conflict with the notion of happens before in the LKMM: any such sequence of writes, RMWs and reads can be converted into a valid sequence of LKMM ppo and rfe relations. This also conveniently encodes the `rmw-sequence` relation used in the LKMM's prop relation.

Compound prop. Finally, we also modified the propagation relation to incorporate `rmw-sequences` through st rather than handling them separately:

$$\mathrm{prop} = (\mathrm{overwrite} \cap \mathrm{ext})^? \fatsemi \mathrm{cumul\text{-}fence} \fatsemi \mathrm{st}^?$$

This way, cumulative fences incorporate $(\mathrm{rf} \fatsemi \mathrm{rmw})^*$ as part of the preceding st relation for fences that allow initial rfe relations (namely po-rel and strong-fence). The terminal $\mathrm{rfe}^?$ of the LKMM's original prop is replaced by $\mathrm{st}^?$, which subsumes it. This reorganization does not change which events can be related through prop on a pure Linux program.

The LKMM's "propagates before" axiom remains the same, acyclic $\mathrm{prop} \fatsemi \mathrm{strong\text{-}fence} \fatsemi \mathrm{hb}^*$, however it uses the new definition of prop and hb.

4.2 Generalized Composition

One key challenge in defining a compound model is allowing C's synchronization patterns while respecting Linux's fence requirements. In order to do so, we redefine relation composition to allow for relations to relate between events and relations rather than only between events. The intuition behind this change in definition is that rather than allowing a synchronization to end in a fence, as C defines synchronization, we allow it to end in the fence plus its entire context. This enables *fence context sharing*, the following synchronization pattern:

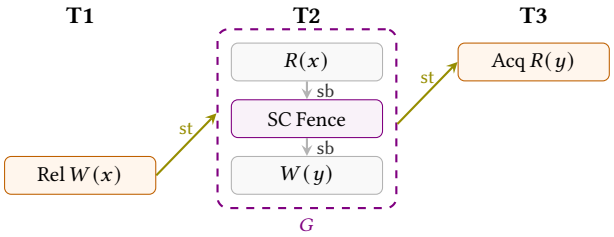

Figure 5. Fence context sharing. The dashed box marks the fence context $G = (R(x), \text{SC Fence}, W(y))$.

Both release and acquire synchronize with the fence and its context, similar to how C would allow synchronization with the fence directly, while Linux fences remain inside their necessary contexts.

To formalize this, we consider a generalization of relations. These are defined as sets of sequences $(s_1, s_2, \ldots, s_n)$ where the s_i can be events or, inductively, other sequences. Relations thus become the special case $\subseteq \{(e_1, e_2) \mid e_1, e_2 \in \text{Events}\}$.

For these generalized relations, we also define a different notion of composition. Let $\mathrm{first}((s_1, \ldots, s_n)) = s_1$ and $\mathrm{last}((s_1, \ldots, s_n)) = s_n$ return the first or last element of the sequence, which may be an event or a sequence. We say two sequences compose when the first or last sequences agree at some level, more formally:

$$\begin{aligned} \mathrm{composes}(s_1, s_2) \text{ iff } & \exists m, n \geq 0, m > 0 \vee n > 0, \\ & \mathrm{last}^m(s_1) = \mathrm{first}^n(s_2) \end{aligned} \tag{1}$$

We then define the composition of generalized relations (sets of sequences) as:

$$r_1 \fatsemi r_2 := \{(s_1, s_2) \in r_1 \times r_2 \mid \mathrm{composes}(s_1, s_2)\} \tag{2}$$

The generalized version allows overlap on intermediate sub-relations—fence contexts—while requiring at least one application of first or last to prevent trivial self-composition.

This mechanism preserves the LKMM's requirement that fences remain in context: the fence never appears as a standalone endpoint. Pattern-dependent fences like `Before_atomic` are only credited with ordering when their required context

(e.g., a following RMW) is part of the composed unit. At the same time, the entire context can participate in multiple synchronizations, recovering C's fence patterns.

The trivial filter. Interpreting the LKMM with generalized relations leads to a subtle yet important distinction when generalizing the identity relation, id:

$$\mathrm{id} = \{s \mid \exists e \in \mathrm{Event}, \mathrm{composes}(s,(e)) \wedge \mathrm{composes}((e),s)\} \quad (3)$$

To understand the issue, consider the following two variants of the message passing litmus, differing only in an acquire being replaced by a read followed by an acquire fence. Both variants should be equivalent, and both behaviors forbidden:

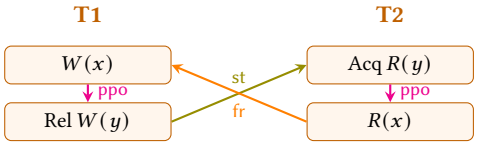

Figure 6. Message passing with an acquire

Variant 1: with an acquire (Figure 6). The prop relation relates $R(x) \xrightarrow{\mathrm{fr}} W(x) \xrightarrow{\mathrm{ppo}} \mathrm{Rel}\, W(y) \xrightarrow{\mathrm{st}} \mathrm{Acq}\, R(y)$, returning to T2. Combined with ppo from $\mathrm{Acq}\, R(y)$ to $R(x)$, this forms a cycle. The prop relation relates $R(x)$ to $\mathrm{Acq}\, R(y)$—these are different events and thus are related by $(\mathrm{prop} \setminus \mathrm{id}) \cap \mathrm{int}$ so the stale read is forbidden.

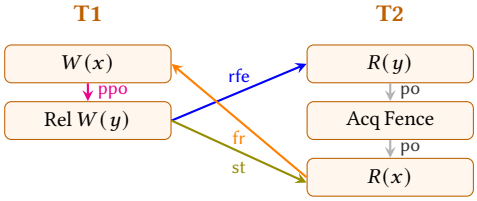

Figure 7. Message passing with an acquire fence

Variant 2: with an acquire fence (Figure 7). The prop relation relates $R(x) \xrightarrow{\mathrm{fr}} W(x) \xrightarrow{\mathrm{ppo}} \mathrm{Rel}\, W(y) \xrightarrow{\mathrm{st}} R(x)$, since the st relation can no longer end at the read of y that has been demoted from an acquire. The prop relation starts and ends on the same event: it is an identity relation. Under the LKMM's $\mathrm{prop} \setminus \mathrm{id}$, this relation would be filtered out, incorrectly permitting the stale read.

To deal with this, we replace the id in $\mathrm{prop} \setminus \mathrm{id}$ with trivial.

$$\mathrm{trivial} = \{(e),(e,e),(e,(e,e)),((e,e),e),(e,e,e),\ldots \mid e \in \mathrm{Event}\} \quad (4)$$

A sequence is trivial if every event in it—at every level of nesting—is the same event. This is strictly stronger than id: the prop edge in Variant 2 traverses fr, ppo, and st, so it starts and ends on the same event but is not trivial. The cycle is preserved and the stale read is correctly forbidden. This change does not affect pure Linux programs.

4.3 Full Model Definition

Having explained the extensions to support C, we are ready to define the compound C-LKMM memory model. The compound model maintains each axiom of the LKMM and adds the no-thin-air axiom specific to C:

Axiom	Definition
Per-location SC	acyclic $(\mathrm{po}_{=\mathrm{loc}} \cup \mathrm{com})$
Atomic RMW	empty $(\mathrm{rmw} \cup (\mathrm{fre} \fatsemi \mathrm{coe}))$
Happens-before	acyclic hb
Propagates-before	acyclic pb
No-thin-air	acyclic $[\mathrm{Plain}] \fatsemi (\mathrm{po} \cup \mathrm{rf})$

where:

$$\begin{aligned}
\text{acq-po} &= \{(a,b) \mid a.str \geq \mathrm{acq}\} \\
&\cup \{(a,f,b) \mid f = \mathrm{Fence}, f.str \geq \mathrm{acq}\} \\
&\cup \{(a,f,g,b) \mid f = \mathrm{Fence}, f.str \geq \mathrm{acq}, g = \mathrm{Fence}\} \quad (5) \\
\text{po-rel} &= \{(a,b) \mid b.str \geq \mathrm{rel}\} \\
&\cup \{(a,f,b) \mid f = \mathrm{Fence}, f.str \geq \mathrm{rel}\} \\
&\cup \{(a,f,g,b) \mid g = \mathrm{Fence}, g.str \geq \mathrm{rel}, f = \mathrm{Fence}\} \quad (6) \\
\text{inter-start} &= [\mathrm{Marked} \cup \mathrm{Rel}] \cup \text{po-rel} \\
&\cup \text{strong-fence} \cup \text{weak-fence} \quad (7) \\
\text{inter-end} &= [\mathrm{Marked} \cup \mathrm{Acq}] \cup \text{acq-po} \\
&\cup \text{strong-fence} \cup \text{weak-fence} \quad (8) \\
\mathrm{st} &= \text{inter-start} \fatsemi (\mathrm{rfe} \cup (\mathrm{rf} \fatsemi \mathrm{rmw})^{+} \fatsemi \mathrm{rf}^{?}) \fatsemi \text{inter-end} \quad (9) \\
\mathrm{hb} &= \mathrm{ppo} \cup \mathrm{st} \cup (\mathrm{prop} \setminus \mathrm{trivial}) \cap \mathrm{int} \quad (10) \\
\mathrm{pb} &= \mathrm{prop} \fatsemi \text{strong-fence} \fatsemi \mathrm{hb}^{*} \quad (11) \\
\mathrm{prop} &= (\mathrm{overwrite} \cap \mathrm{ext})^{?} \fatsemi \text{cumul-fence} \fatsemi \mathrm{st}^{?} \quad (12)
\end{aligned}$$

The relations compose using generalized composition (Equation 2), indicated by the symbol $\fatsemi$. The RC11 no-thin-air axiom is adapted to apply only when every operation in the cycle is a C operation. This is the weakest guarantee needed to match C's behavior and avoids imposing unnecessary restrictions on orderings between Linux operations in compound programs.

5 Soundness

We prove three properties of the compound model: it preserves the behavior of pure C programs (Theorem 1), it preserves the behavior of pure Linux programs (Theorem 2), and the shared synchronization primitives are interchangeable across the boundary (Theorem 3). We provide proof sketches here; full proofs are in Appendix C.

5.1 C Preservation

▶ **Theorem 1** (C Preservation). On a program consisting entirely of C operations (excluding SC atomics), an execution is allowed by the compound model if and only if it is allowed by RC11.

The proof proceeds in two directions.

Forward: compound disallows ⇒ C disallows. We prove any cycle forbidden by the compound model is also forbidden by C. The argument has three parts, one for each axiom.

Happens-before. A cycle in compound hb^+ must map to a violation of C's coherence axiom, $\mathrm{acyclic}(\mathsf{hb}_C \fatsemi \mathsf{eco}^?)$. The key difficulty is that compound hb uses prop, which can introduce multiple non-st communication relations in a single cycle, whereas C's coherence axiom permits at most one. Lemma 5.1 resolves this.

Lemma 5.1 (Prop reduction)**.** Let σ be a cycle in compound hb^+ containing $k \geq 2$ relations from $(\mathsf{prop} \setminus \mathrm{trivial}) \cap \mathrm{int}$. Then either:

(a) σ implies a violation of per-location SC ($\mathrm{acyclic}(\mathsf{po}_{=\mathrm{loc}} \cup \mathsf{com})$), or
(b) there exists cycle σ' in hb^+ with ≤ 1 prop relation.

With Lemma 5.1 in hand, any compound hb cycle with at most one prop relation maps directly to C's coherence axiom. The compound relations translate as follows: $\mathsf{ppo} \subseteq \mathsf{sb}$ (since all of sb is preserved in C), $\mathsf{st} \subseteq (\mathsf{sb} \cup \mathsf{sw})^*$ (since every st between C operations requires release-acquire, which gives sw), and a single prop relation maps to the $\mathsf{eco}^?$ suffix in C's axiom. Composing the above observations gives us that a cycle in compound hb^+ is a cycle in $(\mathsf{sb} \cup \mathsf{sw})^+ \fatsemi \mathsf{eco}^?$, which is exactly what C's coherence axiom forbids.

Propagates-before. A cycle in compound pb^+ must imply a cycle in C's pscf. Rewriting pscf as:

$$[\mathtt{F} \cap \mathtt{SC}]; \mathsf{hb}; (\mathsf{eco}; \mathsf{hb})^?; [\mathtt{F} \cap \mathtt{SC}]$$

Then expanding a cycle constructed from these relations:

$$\ldots; (\mathsf{eco}; \mathsf{hb})^?; [\mathtt{F} \cap \mathtt{SC}]; \mathsf{hb}; (\mathsf{eco}; \mathsf{hb})^?; [\mathtt{F} \cap \mathtt{SC}]; \mathsf{hb}; \ldots$$

We can rewrite the definition of pscf to start and end at a different point in the cycle rather than at the fences:

$$(\mathsf{eco}; \mathsf{hb})^?; [\mathtt{F} \cap \mathtt{SC}]; \mathsf{hb}$$

Of which $\mathsf{prop} \fatsemi \mathsf{strong\text{-}fence} \fatsemi \mathsf{hb}^*$ is a subset since the strong-fence relation implies at least one $\mathsf{po} \subset \mathsf{hb}$ relation before and after the fence.

Per-location SC. A cycle in $\mathrm{acyclic}(\mathsf{po}_{=\mathrm{loc}} \cup \mathsf{com})$ maps directly to C's coherence axiom: $\mathsf{po}_{=\mathrm{loc}} \subseteq \mathsf{sb}$, so the cycle is in $(\mathsf{sb} \cup \mathsf{sw})^+ \fatsemi \mathsf{eco}^?$ with sw vacuous. If the cycle is entirely in com (no $\mathsf{po}_{=\mathrm{loc}}$), then by replacing $\mathsf{rf} \fatsemi \mathsf{fr}$ with co, every write is coherence-ordered before itself, contradicting the definitional irreflexivity of co.

Reverse: C disallows ⇒ compound disallows.

Coherence. A cycle in $(\mathsf{sb} \cup \mathsf{sw})^+ \fatsemi \mathsf{eco}^?$ either contains sw or it does not. If it does not, then this reduces to a cycle in per-loc-SC: every operation in the simplified cycle is on the same location so replace the sb relations with $\mathsf{po}_{=\mathrm{loc}}$. Otherwise for cycles that do contain sw, if the cycle does not contain an eco relation then every thread communicates using sw and thus every po relation can be replaced with po-rel or strong-fence as every thread will have a release or stronger as part of its synchronization with other threads. sw relations map to st and thus the cycle is in compound hb^+. If it does have an eco relation, then this cycle is disallowed via prop: the eco relation is the start of the prop relation, then the cumulative fences of prop are a superset of $(\mathsf{sb} \cup \mathsf{sw})^+$.

PSCF. A cycle in pscf implies a cycle in pb by the same rotation argument: $(\mathsf{eco} \fatsemi \mathsf{hb}_C)^? \subseteq \mathsf{prop}$ and compound hb subsumes C's hb, so $\mathsf{prop} \fatsemi \mathsf{strong\text{-}fence} \fatsemi \mathsf{hb}^*$ captures the rotated cycle.

5.2 LKMM Preservation

▶ **Theorem 2** (LKMM Preservation)**.** On a program consisting entirely of Linux operations, an execution is allowed by the compound model if and only if it is allowed by the LKMM.

Proof sketch. The compound model is defined as an extension of the LKMM, so both directions reduce to subset arguments.
Forward: compound disallows ⇒ LKMM disallows. When restricted to Linux operations, ppo and st reduce to Linux ppo and rfe, and prop either reduces to Linux prop or a per-location SC cycle, and strong-fence is unchanged. Therefore any cycle in compound hb is a cycle in LKMM hb, and any cycle in compound pb is a cycle in LKMM pb.
Reverse: LKMM disallows ⇒ compound disallows. The compound ppo, st, prop, and strong-fence are supersets of their Linux equivalents. Therefore any LKMM hb cycle is a compound hb cycle and any LKMM pb cycle is a compound pb cycle. □

5.3 Interchangeability

The natural formulation of interchangeability—replacing a Linux acquire with a C acquire preserves all allowed behaviors—is too strong. Consider an address dependency between two Linux reads:

$$\text{T1:} \quad r_0 = R(x); \quad \mathrm{Acq}\,R(*r_0)$$

The LKMM preserves the address dependency as ppo regardless of the acquire annotation. If the Linux acquire is replaced by a C acquire, the address dependency is no longer preserved: C does not recognize address dependencies. The behavioral change is not a failure of the acquire primitive—it is a consequence of C's different treatment of intra-thread ordering.

We therefore define interchangeability as preservation of the primitive's own guarantees: its synchronization capability and its ordering contribution.

▶ **Theorem 3** (Interchangeability)**.** For each shared primitive $p \in \{\text{release, acquire, SC fence}\}$:

(a) *Synchronization:* st does not distinguish between a C instance and a Linux instance of p. A C release

synchronizes with any acquire (C or Linux), and vice versa.

(b) *Ordering:* The ordering relations po-rel, acq-po, and strong-fence do not distinguish between a C instance and a Linux instance of p. A C acquire orders subsequent operations via acq-po identically to a Linux acquire.

Proof sketch. Interchangeability follows directly from the construction of the model: the ordering guarantees of an acquire are stated by the acq-po relation, which does not differentiate between Linux and C acquires. The synchronization guarantees of an acquire are defined in st, which does not distinguish between Linux and C acquires. The same is true for releases and SC fences, using po-rel and strong-fence relations for ordering guarantees and st for synchronization guarantees. □

5.4 Model Implementation and Litmus Testing

Implementation. We implemented a Python tool that encodes the compound model and serves as a litmus test engine. Existing tools such as `herd7` [3] and Dartagnan [20] cannot express the generalized fence composition introduced in §4, as it requires chaining more than two events. Our engine enumerates all candidate executions, evaluates the compound model's axioms, and reports the set of allowed outcomes. All litmus results below are produced by this tool, which will be released as part of the artifact.

Litmus testing results. We drew tests from two sources: the C suite from Dartagnan [20], covering Message Passing, Load Buffering, Store Buffering, IRIW, and ISA2; and the LKMM suite distributed with the Linux kernel source tree, including WRC and Coherence. Across all 2,882 tests, the allowed outcomes under the compound model were identical to those of the corresponding single-language model, supporting the C-preservation and LKMM-preservation theorems (§5).

To validate interchangeability, we constructed compound litmus tests by systematically replacing C synchronization primitives with their LKMM counterparts and vice versa: release stores, acquire loads, release fences, acquire fences, and SC fences. All 11,584 resulting tests matched the outcomes of the original, supporting the interchangeability theorem (§5.3). Table 2 summarizes the results.

Suite	# of Tests	Outcomes
C (single-language)	1841	✓
Linux (single-language)	1041	✓
Compound	11584	✓

Table 2. Litmus test validation summary. ✓ = in every test in the suite, the set of allowed outcomes under the compound model is identical to that of the reference model (single-language for C/LKMM rows; original test for compound row).

These 14,466 tests deliver three things: a precise, executable specification of the compound model serving as a reference implementation; empirical support for all three theorems; and a corpus of 11,584 compound litmus tests that gives future cross-boundary verification tools a concrete test target.

6 Case Studies

We demonstrate the compound model's utility both *constructively*—as a design guide for lock-free data structures shared between user-space (C) and kernel-space (LKMM) via eBPF (§6.1)—and *diagnostically*—as a reasoning tool that can expose missing ordering and confirm existing ordering already suffices (§6.2).

6.1 Constructive: eBPF Lock-Free Data Structures

BPF arenas provide regions of memory that both userspace and the kernel can directly access (using real pointers), without a syscall, which would add high overhead. They create a platform for developing shared data structures that span the user-kernel boundary, but they leave open questions about safe synchronization. The formalization of a BPF memory model is ongoing work [5, 16]; we treat eBPF kernel code as LKMM-governed, which is consistent with eBPF's proposed model for all operations our case studies use.

We studied five data structure implementations from widely used concurrent data structure libraries: the CDS Michael-Scott Queue, Facebook Folly's SPSC Queue, ConcurrencyKit's FIFO Queue, Ring, and Stack. Together, they exercise the producer-consumer synchronization patterns that recur throughout user-kernel concurrent code. We present the Michael-Scott Queue in detail below; the remaining four follow the same process and are presented in Appendix D.

For each data structure, we began our work with the reference C/C++ implementation, and apply the **DARE** process:

1. **Distill**: From the reference C implementation, we extract the memory-model agnostic algorithms of the producer (P) and consumer path (C). We use the data structures' invariants to identify the algorithms' required orderings.
2. **Annotate for C**: We re-express P and C as P_C and C_C respectively, annotating each shared-memory access with the C primitive (`relaxed`, `acquire`, `release`) needed to enforce the required orderings under C's `release-acquire`-based happens before.
3. **Relax for LKMM**: We re-express the same algorithms as P_L and C_L. Wherever the LKMM's preserved program order (ppo) already provides a required ordering — through an address dependency, a data dependency, a control-plus-write, or a fence — we drop the corresponding C annotation. The result is a Linux-side implementation in which the programmer makes explicit only the orderings that ppo does not already cover.
4. **Evaluate Heterogeneous Synchronization**: We use the compound model to reason about cross-domain pairings: a kernel-producer paired with a user-consumer, or vice-versa. The compound model's interchangeability property tells us that the shared synchronization primitives carry

the same meaning on both sides and lets us reason about the synchronization.

```
while (retries) {
  tail = load_acq(queue->tail); // C
  tail = load_rlx(queue->tail); // LKMM
  next = load_acq(tail->next);
  if (next != NULL) {
    cmpxchg(queue->tail, tail, next, rel, rlx);
      continue;
  }
  if (cmpxchg(tail->next,
       next, new_node, rel, rlx) == next)
    break;
  continue;
}
if (cmpxchg(queue->tail,
     tail, new_node, rel, rlx) != tail)
  return SUCCESS
return SUCCESS
```

Figure 8. P_C (blue) and P_L (red) for MSQ

```
while (retries) {
  head = load_acq(queue->head); // C
  head = load_rlx(queue->head); // LKMM
  next = load_acq(head->next);
  if (load_acq(queue->head) != head)
    continue;
  if (next == NULL)
    return FAIL;
  tail = load_acq(queue->tail);
  if (head == tail) {
    cmpxchg(queue->tail, tail, next, rel, rlx);
    continue;
  }
  read_data = next->data;
  if (cmpxchg(queue->head,
       head, next, acq, rlx) == head)
    return SUCCESS
  continue;
}
return FAIL
```

Figure 9. C_C (blue) and C_L (red) for MSQ

6.1.1 The Michael-Scott Queue (MSQ).

MSQ [18] is a simple non-blocking queue that uses linked lists. The root node has head and tail pointers, which point to the head and tail nodes respectively. *P* operations insert at the tail (Figure 8), and *C* operations delete from the head (Figure 9).

Invariants.

1. If the `head` and `tail` point to the same address (node), the queue is empty. At initialization, the head and tail point to a dummy node.
2. Insertions always take place at the tail. The `tail` node's next pointer always points to `NULL`. If the `tail` node's next pointer appears to be non-`NULL`, it means there was a racing insertion (the `tail` pointer is lagging behind).
3. Racing insertions are easy to detect (`tail->node.next != NULL`). If these are detected, the insertion algorithm attempts to adjust the tail pointer to the correct tail node and only then performs the insertion (after asserting `tail->node.next == NULL`).
4. Deletions advance the head pointer to skip the current head node, re-pointing it to the next head node.

Required Orderings. *Invariant 2* requires that the new node's `next` pointer be visible (as `NULL`) to any thread that observes it as the new `tail`. *Invariant 3* requires that a consumer that observes the new `tail->next` also observes the corresponding payload write. *Invariant 4* requires that `head` advances only after the new head node is fully linked.

P_C: The C Producer. Figure 8 shows the C-annotated producer, with each annotation mapping to one of the orderings above. The algorithm requires `load_acq(queue->tail)` so the subsequent dereference `tail->next` observes a fully initialized node *(Invariant 2)*. `load_acq(tail->next)` is required since the value read drives a branch and CAS *(Invariant 3)*; without it, C does not order the load against either. The CAS `releases` on `tail->next` and `queue->tail` are required so that the new node's contents — in particular its `NULL` next pointer and its payload — are visible to any future thread that reads the updated `tail` or `tail->next`.

P_L: The LKMM Producer. Figure 8 also shows the LKMM-annotated producer. The C annotation on `load_acq` (queue->tail) becomes `READ_ONCE`: the LKMM's addr-dependency ppo from `tail` to `tail->next` preserves the ordering of the dereference, precisely what *Invariant 2* needs. The CAS `releases` remain, since the LKMM does not provide release semantics for free; they are written as `cas_rel`. The producer thus drops one `acquire` relative to the C version, recovering them through address and control dependencies.

The consumer follows a similar pattern: one acquire is dropped on the LKMM side (`load_acq(queue->head)`) via an addr dependency, and the CAS `release` is preserved.

Evaluate Heterogeneous Pairing P_C with C_L. The most interesting cross-language pairing for MSQ is the user producer paired with the kernel consumer: a user process enqueues events, and a kernel component consumes them. We extract the cross-boundary litmus test and use the compound model to forbid the outcome that would violate *Invariant 3*.

Invariant 3 requires that a consumer observing the new `tail->next` also observes the corresponding payload. Under the compound model, this outcome is ruled out by a single happens-before chain. On the C producer side, there is a `po-rel` edge from the payload write to the `release`. The `st` edge bridges the boundary: the kernel's `cas_rel` satisfies `inter-start` (C release), and the consumer's plain read satisfies `inter-end` (LKMM, no acquire needed). An address-dependency ppo edge from `r0` to `r1` then carries the ordering forward, forbidding the stale read and preserving Invariant 3.

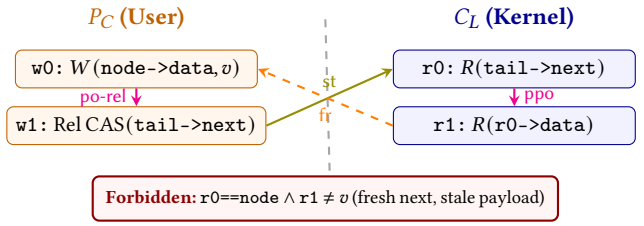

Figure 10. Heterogeneous pairing: user producer (P_C) with kernel consumer (C_L) for the Michael-Scott queue. The st relation bridges the boundary from the user's `cas_rel` to the kernel's `ld_rlx`. $\texttt{w0} \xrightarrow{\text{po-rel}} \texttt{w1} \xrightarrow{\text{st}} \texttt{r0} \xrightarrow{\text{ppo}} \texttt{r1} \xrightarrow{\text{fr}} \texttt{w0}$ is forbidden by the compound model.

The reverse pairing of P_L with C_C is symmetric: the kernel-side `cas_rel` satisfies `inter-start` trivially (LKMM, no release annotation needed), and the C consumer's `load_acq` satisfies `inter-end`, yielding the same happens-before edge via `st`.

Result. All four pairings of MSQ preserve the queue's invariants under the compound model. The matched `release` / `acquire` pattern that closes the KCOV bug from Figure 1 appears here in generalized form: in every cross-boundary pairing, the C side contributes its `release` or `acquire` annotation, the LKMM side contributes ppo, and the compound model's `st` relation joins them across the boundary. The remaining four data structures (Folly SPSC, CK FIFO, CK Ring, CK Stack) follow the same DARE process and are presented in the appendix; Table 3 summarizes all five.

6.1.2 Performance.

To measure performance for both (P_L, C_C) and (P_C,C_L) pairings, our infrastructure allocates two buffers, which we name KU-buffer (for kernel→user communication), and the UK-buffer (for user→kernel communication). For each queue entry, the flow is as follows:

1. **T0**: external syscall (`inode_create`) triggers P_L on KU, producing a single queue entry.
2. **T1**: user-thread polls on KU (C_C) and relays the queue entry to UK (P_C).
3. **T2**: kernel thread consumes the entry from UK (C_L).

We measure the total end-to-end latency for each queue item to ping-pong from kernel mode to user mode and back: $T_{E2E} = T_{P_L} + T_{C_C} + T_{P_C} + T_{C_L}$.

We compare all five structures against `io_uring`/`liburing` as a state-of-the-art baseline, applying the DARE process to extract its submission- and completion-queue paths as (P_L,C_C) and (P_C,C_L) pairings; we strip options unrelated to the core producer-consumer path to ensure a fair comparison. All experiments run on the hardware and software configuration described in Table 4; results are presented in Table 3.

The CK FIFO Queue, Facebook's Folly SPSC Queue, and the CK Ring achieve 8%-12% speedup compared to the adapted `io_uring` implementation. This shows that compound-model-guided custom data structures over BPF Arenas can deliver performance comparable to hand-tuned production structures.

6.2 Diagnostic: Cross-Boundary Analysis

The compound model is also useful for reasoning about existing cross-boundary code. We have already seen one example: the `kcov_remote` bug (§1, Figure 1), where the model exposed a missing write barrier on the kernel side and a missing acquire on the user side, both of which were required to close the happens-before chain. We now apply the same reasoning to a second case where the conclusion is the opposite: the existing code is already correct.

6.2.1 The io_uring Completion Ring: A Non-Bug.

The io_uring completion queue shares a ring buffer between kernel and user space. The kernel writes CQEs and publishes them by advancing a tail index with a release store. The user-space consumer reads the ring as follows:

```
r0:  tail = smp_load_acquire(ring->cq.ktail);
r1:  head = *ring->cq.khead;
     if (head != tail)
r2:    cqe = ring->cq.cqes[(head & mask)];
```

There is an address dependency from `r1` to `r2`, and C does not enforce ordering for address dependencies. We initially submitted a patch promoting `r1` to `smp_load_acquire()`; the patch was accepted upstream. On closer analysis using the compound model, we determined that the original code was already correct (Figure 11).

The critical observation is that `khead` is written only by the user thread to mark consumed entries; the kernel reads it for free-space accounting. On the consumer path, the user's load of `khead` reads its own prior write—so the address dependency from `r1` to `r2` is off the synchronization critical path. The actual cross-boundary synchronization is entirely on `ktail`: the kernel's release pairs with the user's acquire via st, giving the hb chain shown in the figure. The acquire on `khead` was unnecessary.

This case complements `kcov`. In `kcov`, *both* loads in the dependency chain read kernel data—the dependency *is* the critical path, and both fixes were needed (§1). In io_uring, `r1` reads the user's own state, so the dependency is off the critical path and the existing acquire on `ktail` already suffices. The compound model's value lies in both diagnosing missing ordering *and* confirming that existing code is correct.

7 Related Work

Language-Level Memory Models. The Java Memory Model was the first language-level memory model specification [14]. The C/C++11 standard followed with a richer design featuring relaxed, release-acquire, and sequentially consistent atomics; Batty et al. gave the first formal semantics [4], and Lahav et al. repaired the SC axiom to produce RC11 [13], which forms the

DS	P_C	P_L	C_C	C_L	ppo	E2E lat.	Mops/s
Folly SPSC	1 acq 1 rel	1 rel	1 acq 1 rel	1 acq 1 rel	ctrl	**330 ns**	**3.04**
CK Ring SPSC	1 acq 1 rel	1 rel	1 acq 1 rel 1 mb	1 acq 1 rel	acq	**334 ns**	**3.00**
CK FIFO SPSC	1 rel	1 rel	1 acq 1 rel	-	addr	**339 ns**	**2.96**
CK Stack UPMC	1 rel-CAS	1 rel-CAS	1 acq 1 acq-CAS	1 rlx-CAS	addr and data	565 ns	1.77
MSQueue	2 acq 3 rel-CAS	1 acq 3 rel-CAS	4 acq 1 rel-CAS 1 acq-CAS	3 acq 1 rel-CAS 1 rlx-CAS	addr and ctrl	926 ns	1.08
io_uring/liburing	-	-	-	-		369 ns	2.71

Table 3. Synchronization annotations across the four implementations of each data structure, along with end-to-end latency and throughput (averaged from 1000 runs). CK FIFO, Folly, and CK Ring all surpass the Linux `io_uring` performance.

Component	Specification
Processor	Intel Core i5-7600 @ 3.50 GHz (Max 4.10 GHz)
Topology	4 Physical Cores (No SMT), Single NUMA Node
Caches	128 KiB L1, 1 MiB L2, 6 MiB L3
Memory	16 GB DDR4-2400 (Single-Channel)
OS Kernel	Linux v6.18.2
Compiler	Clang/LLVM 20.1.8
BPF Toolchain	bpftool v7.5.0, libbpf v1.5

Table 4. Hardware and software configuration

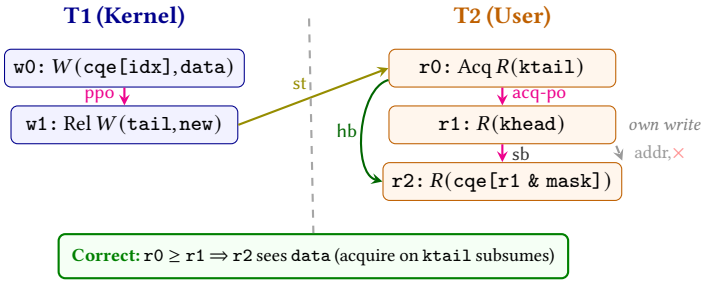

Figure 11. The io_uring completion queue under the compound model. The kernel writes a CQE and releases the tail (`w0`–`w1`). The user acquires the tail (`r0`), reads the head (`r1`, its own bookkeeping), and dereferences (`r2`). The hb chain `w0` $\xrightarrow{\text{ppo}}$ `w1` $\xrightarrow{\text{st}}$ `r0` $\xrightarrow{\text{hb}}$ `r2` guarantees `r2` sees the kernel's write. The address dependency from `r1` to `r2` (greyed, ×) is formally redundant.

basis of the current C standard [11]. These efforts target a single language in isolation; our work addresses what happens when two such models must coexist on shared memory.

The Linux Kernel Memory Model. Howells and McKenney's informal treatment of kernel memory ordering [10, 15] laid the groundwork for the LKMM, which was formalized by Alglave et al. as an axiomatic model in the `cat` language [2]. McKenney has also considered how LKMM primitives map to C atomics [17], observing that many kernel accessors have C11 counterparts. Our work is the first to formalize this interface. The Rust-for-Linux effort defines Rust atomics to be deliberately LKMM-faithful—`Relaxed`, for instance, carries address/data/control dependencies, unlike C `Relaxed` [6]. Our compound model's LKMM basis is therefore consistent with Rust-for-Linux semantics, suggesting applicability to Rust-kernel ↔ user-space C boundaries

Compound Memory Models. The concept of compound memory models—formal composition of distinct consistency specifications over shared memory—was introduced in the Primer [19]. Goens et al. gave the first full formalization for composing hardware-level ISA memory models [8]. These efforts operate at the hardware level; in contrast, our work is the first to compose a language-level model (C) with an OS-level model (LKMM)—two specifications with structurally incompatible notions of happens-before that coexist at the software level.

Formal Tools. Alglave et al.'s `herd7` tool and the `cat` language [3] enable axiomatic memory model specification and litmus test evaluation. For verification of concurrent C programs, Dartagnan [7, 20] performs bounded model checking against axiomatic models, and GenMC [12], Nidhugg [1] provides stateless model checking for C/C++ concurrency. These tools currently target single-model programs; adapting them to check cross-boundary programs against the compound model is a direction for future work.

8 Conclusion

We presented the C/LKMM compound memory model, which composes the C and Linux kernel memory models into a single compound model that preserves each model's semantics in isolation and bridges them at shared-memory boundaries. The model has both diagnostic and constructive value. Applied to existing kernel code, it exposed a real concurrency bug in `kcov` (fix accepted upstream). Applied to new designs, it guided the construction of five cross-boundary lock-free data structures over eBPF.

As eBPF, io_uring, and similar zero-copy interfaces continue to blur the boundary between user space and the kernel, the need for formal cross-boundary reasoning will only grow. Our experience suggests that composability—the ability to bridge two models at a shared boundary—warrants consideration as a design criterion alongside correctness and performance in future memory model work. Our compound model provides the foundation for that reasoning.

9 Acknowledgements

Generative AI tools (including ChatGPT Codex, Claude, and GitHub Copilot) were used to assist the development of the Compound Model litmus testing engine (§5.4), and the eBPF heterogenous data structure framework (§6). In addition, AI tools were also used to code the diagrams in this paper and copyedit the text.

References

[1] Parosh Aziz Abdulla, Stavros Aronis, Mohamed Faouzi Atig, Bengt Jonsson, Carl Leonardsson, and Konstantinos Sagonas. 2015. Stateless Model Checking for TSO and PSO. In *Tools and Algorithms for the Construction and Analysis of Systems*, Christel Baier and Cesare Tinelli (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 353–367.

[2] Jade Alglave, Luc Maranget, Paul E. McKenney, Andrea Parri, and Alan S. Stern. 2018. Frightening Small Children and Disconcerting Grown-ups: Concurrency in the Linux Kernel. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2018, Williamsburg, VA, USA, March 24-28, 2018*. ACM, 405–418. https://doi.org/10.1145/3173162.3177156

[3] Jade Alglave, Luc Maranget, and Michael Tautschnig. 2014. Herding Cats. *ACM TOPLAS* 36, 2 (jul 2014), 1–74. https://doi.org/10.1145/2627752

[4] Mark Batty, Scott Owens, Susmit Sarkar, Peter Sewell, and Tjark Weber. 2011. Mathematizing C++ concurrency. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2011, Austin, TX, USA, January 26-28, 2011*, Thomas Ball and Mooly Sagiv (Eds.). ACM, 55–66. https://doi.org/10.1145/1926385.1926394

[5] Jonathan Corbet. 2024. A memory model for BPF. LWN.net. https://lwn.net/Articles/976071/.

[6] Boqun Feng. 2026. [PATCH 0/2] Provide Rust atomic helpers over raw pointers. https://lore.kernel.org/all/20260120115207.55318-1-boqun.feng@gmail.com/

[7] Natalia Gavrilenko, Hernán Ponce de León, Florian Furbach, Keijo Heljanko, and Roland Meyer. 2019. BMC for Weak Memory Models: Relation Analysis. In *Computer Aided Verification - 31st International Conference, CAV 2019, New York City, NY, USA, July 15-18, 2019, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 11561)*, Isil Dillig and Serdar Tasiran (Eds.). Springer, 355–373. https://doi.org/10.1007/978-3-030-25540-4_19

[8] Andrés Goens, Soham Chakraborty, Susmit Sarkar, Sukarn Agarwal, Nicolai Oswald, and Vijay Nagarajan. 2023. Compound memory models. *Proceedings of the ACM on Programming Languages* 7, PLDI (2023), 1145–1168.

[9] Paul Heidekrüger and M Elver. 2022. Status Report: Broken Dependency Orderings in the Linux Kernel.

[10] David Howells, Paul E. McKenney, Will Deacon, and Peter Zijlstra. 2024. memory-barriers.txt. Linux kernel documentation, https://www.kernel.org/doc/Documentation/memory-barriers.txt.

[11] International Organization for Standardization 2024. *ISO/IEC 9899:2024 Information technology — Programming languages — C*. International Organization for Standardization, Geneva, Switzerland.

[12] Michalis Kokologiannakis, Azalea Raad, and Viktor Vafeiadis. 2019. Model checking for weakly consistent libraries. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019, Phoenix, AZ, USA, June 22-26, 2019*, Kathryn S. McKinley and Kathleen Fisher (Eds.). ACM, 96–110. https://doi.org/10.1145/3314221.3314609

[13] Ori Lahav, Viktor Vafeiadis, Jeehoon Kang, Chung-Kil Hur, and Derek Dreyer. 2017. Repairing sequential consistency in C/C++11. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, Barcelona, Spain, June 18-23, 2017*, Albert Cohen and Martin T. Vechev (Eds.). ACM, 618–632. https://doi.org/10.1145/3062341.3062352

[14] Jeremy Manson, William Pugh, and Sarita V. Adve. 2005. The Java memory model. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2005, Long Beach, California, USA, January 12-14, 2005*, Jens Palsberg and Martín Abadi (Eds.). ACM, 378–391. https://doi.org/10.1145/1040305.1040336

[15] Paul E. McKenney. 2017. *A formal kernel memory-ordering model (part 1)*. https://lwn.net/Articles/718628/

[16] Paul E. McKenney and Puranjay Mohan. 2024. Towards a BPF Memory Model. Linux Plumbers Conference. https://lpc.events/event/18/contributions/1949/.

[17] Paul E. McKenney, Ulrich Weigand, Andrea Parri, and Boqun Feng. 2016. Linux-Kernel Memory Model. http://open-std.org/JTC1/SC22/WG21/docs/papers/2016/p0124r2.html. ISO/IEC JTC1/SC22/WG21 P0124R2.

[18] Maged M Michael and Michael L Scott. 1996. Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms. In *Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computing*. 267–275.

[19] Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, and David A. Wood. 2020. *A Primer on Memory Consistency and Cache Coherence* (2 ed.). Morgan & Claypool Publishers.

[20] Hernán Ponce-de León, Florian Furbach, Keijo Heljanko, and Roland Meyer. 2020. Dartagnan: Bounded Model Checking for Weak Memory Models (Competition Contribution). In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 378–382.

A Formal Model Primitives

This appendix defines the primitive objects on which the formal model and proofs rest: event fields, the event structure and its restrictions, the standard axiomatic relations, the generalized composition operator, and the definition of programs and executions.

A.1 Linux Fence Types

Linux fences can have one of the following types. C's sequentially consistent fences are classified as "MB", and C's other fences use release, acquire, and acquire-release fence strengths.

$$\begin{aligned} \mathit{FenceType} = {} & \mathit{Wmb} \mid \mathit{Rmb} \mid \mathit{Mb} \mid \mathit{Barrier} \\ & \mid \mathit{Rcu\text{-}lock} \mid \mathit{Rcu\text{-}unlock} \mid \mathit{Sync\text{-}rcu} \\ & \mid \mathit{Before\text{-}atomic} \mid \mathit{After\text{-}atomic} \\ & \mid \mathit{After\text{-}spinlock} \mid \mathit{After\text{-}unlock\text{-}lock} \end{aligned}$$

A.2 Event Types

Event types include:

$$\mathit{EventType} = \{\mathsf{R},\mathsf{W},\mathsf{RMW},\mathsf{F}\}$$

A.3 Language

$$\mathit{Language} = \{\mathsf{C},\mathsf{Linux}\}$$

A.4 Strength

$$\mathit{Strength} = \{\textsc{rlx},\textsc{rel},\textsc{acq},\textsc{aqrl},\{\mathit{FenceType}\}\}$$

A.5 Event Structure

Events contain the following attributes:

$$\begin{aligned} \mathit{thread} &: \mathbb{Z} \\ \mathit{thread\_pos} &: \mathbb{Z} \\ \mathsf{loc} &: \mathbb{Z} \\ \mathit{lang} &: \mathit{Language} \\ \mathit{event\_type} &: \mathit{EventType} \\ \mathit{str} &: \mathit{Strength} \end{aligned}$$

Event restrictions. There is no C write with strength Before-atomic, for example. These restrictions match the existing allowed event language, type, and strength combinations for C and Linux.

$$\begin{aligned} \forall e : \mathit{Event}, e.\mathsf{lang} = \mathsf{C} \land e.\mathsf{event\_type} = \mathsf{W} \\ \implies e.\mathit{str} \in \{\textsc{rlx},\textsc{rel},\textsc{mb}\} \\ \vdots \end{aligned}$$

Notably allowed is the LKMM's MB RMW but not C11's MB RMW, as the behavior of C11's MB RMW is determined by the pscb relation of the PSC axiom which has not been formalized in this compound model.

A.6 Relations

See the relation composition section (§A.7) for definitions of *initial* and *terminal*.

Internal.

$$\begin{aligned} \mathsf{int} = \{ s = (s_0,s_1,\ldots,s_n) \mid {} & \\ & (\forall i, s_i : \mathit{Event} \cup \mathit{Relation}) \\ & \land \mathit{initial}(s).\mathit{thread} \\ & \quad = \mathit{terminal}(s).\mathit{thread} \} \end{aligned}$$

External.

$$\begin{aligned} \mathsf{ext} = \{ s = (s_0,s_1,\ldots,s_n) \mid {} & \\ & (\forall i, s_i : \mathit{Event} \cup \mathit{Relation}) \\ & \land \mathit{initial}(s).\mathit{thread} \\ & \quad \neq \mathit{terminal}(s).\mathit{thread} \} \end{aligned}$$

Program order (po).

$$\begin{aligned} \mathsf{po} = \{ (e_0,e_1) \mid {} & (e_0,e_1) \in \mathsf{int} \\ & \land e_0.\mathit{thread\_pos} < e_1.\mathit{thread\_pos} \} \end{aligned}$$

Communication relations. Communication relations are given as part of an execution, with additional sanity checks so that we can infer certain properties about communication in proofs (e.g., a read only reads from a single write). There is nothing inherent to an rf that says it must agree with po order, which is why this is an axiom of the memory model and not part of the definition of an rf.

Reads-from (rf). The last condition states a read reads from a unique write.

$$\begin{aligned} \mathsf{rf} \subseteq \{ (e_0,e_1) \mid {} & (\forall i, e_i : \mathit{Event}) \\ & \land e_0.\mathsf{event\_type} = \mathsf{W} \\ & \land e_1.\mathsf{event\_type} = \mathsf{R} \\ & \land e_0.\mathsf{loc} = e_1.\mathsf{loc} \\ & \land ((e_0,e_1) \in \mathsf{rf} \land (e_0',e_1) \in \mathsf{rf} \\ & \quad \implies e_0 = e_0') \} \end{aligned}$$

Coherence order (co). The last condition is transitivity.

$$\begin{aligned} \mathsf{co} \subseteq \{ (e_0,e_1) \mid {} & (\forall i, e_i : \mathit{Event}) \\ & \land e_0.\mathsf{event\_type} = \mathsf{W} \\ & \land e_1.\mathsf{event\_type} = \mathsf{W} \\ & \land e_0.\mathsf{loc} = e_1.\mathsf{loc} \\ & \land ((a,b) \in \mathsf{co} \land (b,c) \in \mathsf{co} \\ & \quad \implies (a,c) \in \mathsf{co}) \} \end{aligned}$$

From-read (fr).

$$\begin{aligned}\mathsf{fr} = \{(e_0,e_1,\ldots,e_n) \mid (\forall i, e_i : \mathit{Event}) \\ \wedge \exists j, 0 < j < n, \\ (e_j,\ldots,e_0) \in \mathsf{rf}, \\ (e_j,\ldots,e_n) \in \mathsf{co}\}\end{aligned}$$

Overwrite.

$$\mathsf{overwrite} = \mathsf{co} \cup \mathsf{fr}$$

Communication and extended communication.

$$\mathsf{com} = \{(e_0,e_1,\ldots,e_n) \mid (e_0,e_1,\ldots,e_n) \in (\mathsf{rf} \cup \mathsf{co} \cup \mathsf{fr})\}$$

$$\mathsf{eco} = \mathsf{com}^+$$

A.7 Relation Composition

In herd, the semicolon is used for relation composition (e.g., ppo$\fatsemi$rfe is a ppo followed by an rfe) where the two relations can overlap on a single event. We define composition such that it also allows relations to overlap on an entire relation; for example, an SC fence acting as both release and acquire requires that it can be seen by both the release-acquire pattern going to it and the release-acquire pattern going out of it.

For composing a relation between events with a relation between relations (i.e., fences), we need to be able to see if the overlap is on the appropriate event in the fence. If the fence is second in the composition, the relation must overlap on either the entire fence or on the first event of the fence. Similarly, if the fence is first, it must overlap on the entire fence or on the last event of the fence. We generalize this to relations that are compositions of relations themselves, such as prop, by allowing the application of *first* and *last* as many times as necessary, so long as at least one is applied once (otherwise any relation composes with itself since $\mathit{first}^0((a,b)) = \mathit{last}^0((a,b))$).

For this reason, we define functions to reveal the first and last events in a relation:

$$\mathit{first}(r) := \begin{cases} r & \text{if } r \in \mathit{Event} \\ r_0 & \text{if } r = (r_0,r_1,\ldots,r_n) \end{cases}$$

$$\mathit{last}(r) := \begin{cases} r & \text{if } r \in \mathit{Event} \\ r_n & \text{if } r = (r_0,r_1,\ldots,r_n) \end{cases}$$

initial and *terminal* return the first or last *event* in a relation rather than potentially the first or last relation:

$$\mathit{initial}(r) := \begin{cases} r & \text{if } r \in \mathit{Event} \\ \mathit{initial}(r_0) & \text{if } r = (r_0,r_1,\ldots,r_n) \end{cases}$$

$$\mathit{terminal}(r) := \begin{cases} r & \text{if } r \in \mathit{Event} \\ \mathit{terminal}(r_n) & \text{if } r = (r_0,r_1,\ldots,r_n) \end{cases}$$

Generalized composition. We say two sequences compose when the first or last sequences agree at some level, more formally:

$$\begin{aligned}\mathrm{composes}(s_1,s_2) \text{ iff } \exists m,n \geq 0, m > 0 \vee n > 0, \\ \mathrm{last}^m(s_1) = \mathrm{first}^n(s_2)\end{aligned} \tag{13}$$

We then define the composition of generalized relations (sets of sequences) as:

$$r_1 \fatsemi r_2 := \{(s_1,s_2) \in r_1 \times r_2 \mid \mathrm{composes}(s_1,s_2)\} \tag{14}$$

The original definition of relation composition, where relations agree on a single event rather than an event or relation, is defined as:

$$r_1 \fatsemi r_2 := \{(s_1,s_2) \in r_1 \times r_2 \mid \mathit{terminal}(r_1) = \mathit{initial}(r_2)\} \tag{15}$$

Acyclicity We define the transitive closure:

$$r^+ = \{(s_1,s_2,\ldots,s_n) \mid (\forall i \leq n, s_i \in r) \wedge (\forall i < n, \mathrm{composes}(s_i,s_{i+1}))\}$$

From this we define acyclicity of r as the irreflexivity of the transitive closure of r, i.e. $\nexists (s_1,s_2,\ldots,s_n) \in r^+, s_1 = s_n$.

A.8 Programs and Executions

Program.

$$\mathit{Program} = \{\{e_0,e_1,\ldots\} \mid \forall i, e_i : \mathit{Event}\}$$

Execution.

$$\mathit{Execution} = \{P : \mathit{Program}, C = \{c_0,c_1,\ldots \mid c_i \in \mathsf{com}\}\}$$

A.9 Notes

- A relation is in trivial if it contains a single event or contains only relations that are trivial (i.e., is closed under transitivity, so $((e_0),(e_0,e_0))$ is trivial):

$$\begin{aligned}\mathsf{trivial} = \{r = (r_0,r_1,\ldots,r_n) \mid \\ \exists e \in \mathit{Event}, \forall i \leq n, \\ r_i \fatsemi (e) \wedge (r_i = e \vee r_i \in \mathsf{trivial})\}\end{aligned}$$

- A relation is in id if it composes with itself; e.g., $((e_0,e_1),(e_1,e_0))$ starts and ends with e_0:

$$\mathsf{id} = \{r \mid r \fatsemi r\}$$

B Formal Model Relations

This appendix gives the complete formal definitions of all compound model relations, corresponding to the informal development in §4.

B.1 Basic Relations

Release-acquire. A po-rel is a sequence of events such that the first is po-before the last, the last event is a write, and either the last event is a release or there exists a release fence po-between the first and last events. The final two variants allow for a second fence to exist between the first event and the release write or release fence. This allows for an acquire fence followed by a release write or release fence to be in both

po-rel and acq-po, thus allowing the ordering of acquire and release fences that exists in C.

$$\begin{aligned}\text{po-rel} = \{&(e_0,e_1) \mid (e_0,e_1) \in \text{po} \\ &\wedge e_1.\text{event\_type} \neq \text{F} \wedge e_1.str \geq \text{REL} \\ &\wedge e_0.\text{event\_type} \neq \text{F}\} \\ \cup \{&(e_0,e_1,e_2) \mid (e_0,e_1) \in \text{po} \wedge (e_1,e_2) \in \text{po} \\ &\wedge e_1.\text{event\_type} = \text{F} \wedge e_1.str \geq \text{REL} \\ &\wedge e_2.\text{event\_type} \in \{\text{W},\text{RMW}\} \\ &\wedge e_0.\text{event\_type} \neq \text{F}\} \\ \cup \{&(e_0,e_1,e_2) \mid (e_0,e_1) \in \text{po} \wedge (e_1,e_2) \in \text{po} \\ &\wedge e_1.\text{event\_type} = \text{F} \\ &\wedge e_2.\text{event\_type} \neq \text{F} \wedge e_2.str \geq \text{REL} \\ &\wedge e_0.\text{event\_type} \neq \text{F}\} \\ \cup \{&(e_0,e_1,e_2,e_3) \mid \\ &(e_0,e_1) \in \text{po} \wedge (e_1,e_2) \in \text{po} \wedge (e_2,e_3) \in \text{po} \\ &\wedge e_1.\text{event\_type} = \text{F} \wedge e_2.event\_type = \text{F} \\ &\wedge e_2.str \geq \text{REL} \\ &\wedge e_3.\text{event\_type} \in \{\text{W},\text{RMW}\} \\ &\wedge e_0.\text{event\_type} \neq \text{F}\} \end{aligned} \tag{16}$$

The acq-po relation is defined similarly.

$$\begin{aligned}\text{acq-po} = \{&(e_0,e_1) \mid (e_0,e_1) \in \text{po} \\ &\wedge e_0.\text{event\_type} \neq \text{F} \wedge e_0.str \geq \text{ACQ} \\ &\wedge e_1.\text{event\_type} \neq \text{F}\} \\ \cup \{&(e_0,e_1,e_2) \mid (e_0,e_1) \in \text{po} \wedge (e_1,e_2) \in \text{po} \\ &\wedge e_1.\text{event\_type} = \text{F} \wedge e_1.str \geq \text{ACQ} \\ &\wedge e_0.\text{event\_type} \in \{\text{R},\text{RMW}\} \\ &\wedge e_2.\text{event\_type} \neq \text{F}\} \\ \cup \{&(e_0,e_1,e_2) \mid (e_0,e_1) \in \text{po} \wedge (e_1,e_2) \in \text{po} \\ &\wedge e_0.\text{event\_type} \neq \text{F} \wedge e_0.str \geq \text{ACQ} \\ &\wedge e_1.\text{event\_type} = \text{F} \\ &\wedge e_2.\text{event\_type} \neq \text{F}\} \\ \cup \{&(e_0,e_1,e_2,e_3) \mid \\ &(e_0,e_1) \in \text{po} \wedge (e_1,e_2) \in \text{po} \wedge (e_2,e_3) \in \text{po} \\ &\wedge e_1.\text{event\_type} = \text{F} \wedge e_1.str \geq \text{ACQ} \\ &\wedge e_0.\text{event\_type} \in \{\text{R},\text{RMW}\} \\ &\wedge e_2.\text{event\_type} = \text{F} \\ &\wedge e_3.\text{event\_type} \neq \text{F}\} \end{aligned} \tag{17}$$

Fences. The strong-fence relation includes all LKMM strong-fence fence relations as previously defined, as well as C11's SC fence. This will also include the RMW tagged as MB, which functions like a strong-fence (i.e. interacts with pb) despite not explicitly being a fence itself.

$$\begin{aligned}\text{rmb} = \{&(e_0,e_1,e_2) \mid \\ &(e_0,e_1) \in \text{po} \wedge (e_1,e_2) \in \text{po} \\ &\wedge e_0.\text{event\_type} = \text{R} \wedge e_1.event\_type = \text{F} \\ &\wedge e_2.\text{event\_type} = \text{R} \wedge e_1.str = Rmb\} \end{aligned} \tag{18}$$

$$\begin{aligned}\text{wmb} = \{&(e_0,e_1,e_2) \mid \\ &(e_0,e_1) \in \text{po} \wedge (e_1,e_2) \in \text{po} \\ &\wedge e_0.\text{event\_type} = \text{W} \wedge e_1.event\_type = \text{F} \\ &\wedge e_2.\text{event\_type} = \text{W} \wedge e_1.str = Wmb\} \end{aligned} \tag{19}$$

$$\begin{aligned}\text{mb} = \{&(e_0,e_1,e_2) \mid \\ &(e_0,e_1) \in \text{po} \wedge (e_1,e_2) \in \text{po} \\ &\wedge e_0.\text{event\_type} \neq \text{F} \wedge e_1.event\_type = \text{F} \\ &\wedge e_2.\text{event\_type} \neq \text{F} \wedge e_1.str = \text{MB}\} \\ \cup \{&(e_0,e_1) \mid (e_0,e_1) \in \text{po} \\ &\wedge e_0.\text{event\_type} \neq \text{F} \\ &\wedge e_1.\text{event\_type} = \text{R} \wedge e_1.str = \text{MB}\} \\ \cup \{&(e_0,e_1) \mid (e_0,e_1) \in \text{po} \\ &\wedge e_0.\text{event\_type} = \text{W} \wedge e_0.str = \text{MB} \\ &\wedge e_1.\text{event\_type} \neq \text{F}\} \\ \cup \{&(e_0,e_1,e_2) \mid \\ &(e_0,e_1) \in \text{po} \wedge (e_1,e_2) \in \text{po} \\ &\wedge e_0.\text{event\_type} \neq \text{F} \wedge e_1.event\_type = \text{F} \\ &\wedge e_2.\text{event\_type} = \text{RMW} \\ &\wedge e_1.str = Before\_atomic\} \\ \cup \{&(e_0,e_1,e_2,e_3) \mid \\ &(e_0,e_1) \in \text{po} \wedge (e_1,e_2) \in \text{po} \wedge (e_2,e_3) \in \text{po} \\ &\wedge e_0.\text{event\_type} \neq \text{F} \wedge e_1.event\_type = \text{F} \\ &\wedge e_2.\text{event\_type} = \text{RMW} \\ &\wedge e_3.\text{event\_type} \neq \text{F} \\ &\wedge e_1.str = Before\_atomic\} \\ \cup \{&(e_0,e_1,e_2) \mid \\ &(e_0,e_1) \in \text{po} \wedge (e_1,e_2) \in \text{po} \\ &\wedge e_0.\text{event\_type} = \text{RMW} \wedge e_1.event\_type = \text{F} \\ &\wedge e_2.\text{event\_type} \neq \text{F} \\ &\wedge e_1.str = After\_atomic\} \\ \cup \{&(e_0,e_1,e_2,e_3) \mid \\ &(e_0,e_1) \in \text{po} \wedge (e_1,e_2) \in \text{po} \wedge (e_2,e_3) \in \text{po} \\ &\wedge e_0.\text{event\_type} \neq \text{F} \\ &\wedge e_1.\text{event\_type} = \text{RMW} \\ &\wedge e_2.\text{event\_type} = \text{F} \wedge e_3.event\_type \neq \text{F} \\ &\wedge e_2.str = After\_atomic\} \end{aligned} \tag{20}$$

$$\text{strong-fence} = \text{mb} \tag{21}$$

C adds a single "weak" fence (AQRL), which is like an SC fence but does not interact with pb / PSC.

B.2 Fundamental Coherence Ordering

Sequential consistency per variable.

$$\text{po}_{=\text{loc}} = \{(e_0,e_1) \mid (e_0,e_1) \in \text{po} \wedge e_0.\text{loc} = e_1.\text{loc}\}$$

$$\text{per-loc-SC} := \text{acyclic}(\text{po}_{=\text{loc}} \cup \text{com})$$

Atomic read-modify-write. In the LKMM this is defined by saying the set of relations that are both rmw and (fre;coe) is

empty.

$$\forall e : Event, e.\text{event\_type} = \text{RMW} \implies \nexists e' : Event, (e,e') \in (\text{fr} \cap \text{ext}) \wedge (e',e) \in (\text{co} \cap \text{ext})$$

B.3 Instruction Execution Ordering

Preserved program order (ppo). The ppos of the compound model are the LKMM ppos, interpreted under the new generalized relation composition, plus the C additions made to acq-po, po-rel, and fences.

Propagation: ordering from release operations and strong fences. Similar to Linux, this is defined as overwrite∩ext, followed by cumulative fences (e.g., release or stronger), followed by a terminal rfe. For C, this is modified such that any rfe that takes place inside of the relation are release-acquire as far as C is concerned. Each cumulative fence allows a preceding rfe after the initial overwrite, excluding po-unlock-lock-po. This is true even for wmb, because $[A]\,;\text{overwrite};[B];\text{rfe};[C];\text{wmb};[D]$ implies C is a RMW and thus $[A];\text{overwrite};[C]$.

$$\text{prop} = \{(s_0,s_1,s_2) \mid s_0 \in (\text{overwrite} \cap \text{ext})^? \wedge s_1 \in \text{cumul-fences} \wedge s_2 \in \text{st}^? \wedge \text{composes}(s_0,s_1) \wedge \text{composes}(s_1,s_2)\} \tag{22}$$

Cumulative fences are defined to include the preceding rfe (when allowed by Linux), as well as the rmw-sequence that can follow any cumul-fence; however, this rmw-sequence has been shifted to the beginning of a cumul-fence as well as included in the final st which replaces the final rfe.

Fences that allow a preceding rfe use st at their start; fences that do not use inter-start;rmw-sequence;inter-end to avoid the rfe; and wmb is further restricted to not allow acquire fences in inter-end.

$$\begin{aligned}\text{cumul-fence} = \{&(s_0,s_1) \mid s_0 \in \text{st} \wedge s_1 \in \text{strong-fence} \cup \text{po-rel} \wedge \text{composes}(s_0,s_1)\} \\ \cup \{&(s_0,s_1) \mid s_0 \in (\text{inter-start},\text{rmw-sequence},\text{inter-end}) \wedge s_1 \in \text{po-unlock-lock-po} \wedge \text{composes}(s_0,s_1)\} \\ \cup \{&(s_0,s_1) \mid s_0 \in (\text{inter-start},\text{rmw-sequence}, (\text{inter-end} \setminus \text{acq-po}) \wedge s_1 \in \text{wmb}) \wedge \text{composes}(s_0,s_1)\}\end{aligned} \tag{23}$$

Here it is defined to be weak enough to admit any observable behavior, and could potentially be strengthened, for example depending on how a wmb interacts with a non-volatile write in terms of cumulativity (i.e. adding wmb to inter-start).

$$\text{cumul-fences} = (\text{rfe};(\text{cumul-fence} \setminus \text{po-unlock-lock-po}) \mid \text{po-unlock-lock-po})^?;\text{cumul-fence}^* \tag{24}$$

Synchronize threads (st). This is how we define inter-thread synchronization patterns, where Linux uses writes and reads but C uses releases and acquires.

$$\text{inter-start} = \{(e) \mid e \in Event \cup Relation \wedge (e.lang = \text{Linux} \vee e.str \geq \text{REL} \vee e \in \text{po-rel} \cup \text{strong-fence} \cup \text{weak-fence})\} \tag{25}$$

$$\text{inter-end} = \{(e) \mid e \in Event \cup Relation \wedge (e.lang = \text{Linux} \vee e.str \geq \text{ACQ} \vee e \in \text{acq-po} \cup \text{strong-fence} \cup \text{weak-fence})\} \tag{26}$$

$$\text{rmw-sequence} = (\text{rf};\text{rmw})^+ \tag{27}$$

$$\text{st} = \{(e_0,e_1,e_2) \mid (\forall i, e_i \in Event \cup Relation) \wedge e_0 \in \text{inter-start} \wedge e_2 \in \text{inter-end} \wedge e_1 \in (\text{rfe} \cup (\text{rmw-sequence};\text{rf}^?))\} \tag{28}$$

Happens before. Similar to Linux, we take the transitive closure of ppo, st (in place of rfe), and (prop \ trivial)∩int.

$$\text{hb} = (\text{ppo} \cup \text{st} \cup ((\text{prop} \setminus \text{trivial}) \cap \text{int}))$$
$$\text{acyclic}(\text{hb})$$

B.4 Write and Fence Propagation Ordering

Propagation: each non-rf link needs a strong fence.

$$\text{pb} = \text{prop};\text{strong-fence};\text{hb}^*$$
$$\text{acyclic}(\text{pb})$$

C Proofs

Appendix A defines the primitive objects—events, relations, and generalized composition—that the model and proofs depend on. Appendix B gives the complete formal definitions of all compound model relations, corresponding to §4.

The following sections contain proofs for: C-compound model equivalence, LKMM-compound model equivalence, and interchangeability.

Table 5 maps each claim in §5 to its full proof below.

C.1 Equivalence of C and the Compound Model.

The following proofs correspond with Section 5 Theorem 1.

There are two parts to proving that the compound model is no stronger than C: any po edge is already ordered by C, so there is nothing to show here, but what we need to show is that any interaction between threads is either release-acquire (or stronger) or eco, and that there is only a single eco edge in any happens-before cycle.

More specifically, given that the compound happens-before axiom is defined as the acyclicity of three relations (ppo, st,

Memory Model	Section 5	Direction	Appendix C
C	Lemma 5.1 (prop Reduction)	–	§C.1.1
C	Theorem 1 (hb Equivalence)	Forward (Compound to C)	§C.1.2
C	Theorem 1 (hb Equivalence)	Reverse (C to Compound)	§C.1.4
C	Theorem 1 (per-loc SC)	–	§C.1.5
C	Theorem 1 (pb Equivalance)	Both	§C.1.6
LKMM	Theorem 2 (hb Equivalence)	Both	§C.2.1
LKMM	Theorem 2, (pb Equivalence)	Both	§C.2.2
Interchange-ability	Theorem 3	–	§C.3

Table 5. Correspondence between Section 5 theorems and Appendix C proofs.

$(\mathsf{prop} \setminus \mathsf{trivial}) \cap \mathsf{int})$, we need to show that: ppo is either part of C's sb or sw (and it is, since every ppo is a po edge = sb edge); st is part of sb or sw (and for C it is release-acquire or stronger, so it is a subset of sw since sw allows release-acquire synchronization to be internal to a thread compared to st's explicitly external release-acquire); and prop is the special case that uses eco so we show prop is only used once in a cycle (thus only one eco appears) and prop is a subset of $\mathsf{eco}^?;(\mathsf{sb} \cup \mathsf{sw})^+$. Notably only a subset thanks to internal eco and internal synchronizations allowed in C.

Lemma C.1 shows that only a single eco edge is ever used in a disallowed compound happens-before cycle comprised of only C operations by showing that any cycle of C operations disallowed by the compound happens-before that uses multiple prop edges (and thus multiple eco edges) either implies the existence of another compound happens-before cycle that uses only a single prop edge or that the execution is disallowed by per-location-SC. In this way, the execution would have been disallowed by a valid C happens-before cycle anyway.

C.1.1 C Preservation: Prop Reduction.

The following is a helper Lemma that is used in the forward direction proof of the equivalence of the compound model and the C memory model.

Lemma C.1. The compound happens-before on a C program does not disallow cycles with multiple prop edges (and thus loops with multiple eco edges) unless the execution is already disallowed by a happens-before cycle with at most one prop edge or is disallowed by per-location-SC. This corresponds with Section 5 Lemma 5.1.

$$
\begin{aligned}
&\forall E = (P, C) : \mathit{Execution}, \\
&\quad (\forall a \in P, a.\mathit{lang} = \mathrm{C}) \\
\implies &\big((\exists r_i \subset P, \\
&\qquad r = (r_0, \ldots, r_n = r_0) \in \mathsf{hb}^+\big) \\
\implies &(\exists s_i \subset P, s = (s_0, \ldots, s_m = s_0), \\
&\qquad \big((s \in \mathsf{hb}^+ \\
&\qquad \wedge ((\forall j, 0 \le j \le m, \\
&\qquad\quad s_j \in (\mathsf{ppo} \cup \mathsf{st})) \\
&\qquad \vee (\exists! j, 0 \le j \le m, \\
&\qquad\quad s_j \notin (\mathsf{ppo} \cup \mathsf{st})))) \\
&\qquad \vee s \in (\mathsf{po}_{=\mathsf{loc}} \cup \mathsf{com})^+)\big)\big)
\end{aligned}
$$

In words, the theorem says: given an execution, if the program contains only C operations, then if there exists a compound happens-before cycle in the execution, there must exist a cycle in the execution with one of the following properties: it is a happens-before cycle with at most one prop edge, or it is a per-loc-SC cycle. We write it as the s_j not in ppo ∪ st because we want to limit s_l to prop relations that are not also ppo or st (since po-rel is in both ppo and prop, for example).

Proof overview.

If there is already one or fewer prop in the cycle, we are done.

Assume there are two or more prop edges. Any prop edge that does not begin with an eco edge is equivalent to a sequence of ppo and st and can immediately be simplified to a cycle with one fewer prop edges, so assume all prop edges begin with eco (implicitly by removing all non-eco-starting prop edges).

Consider one of the prop edges: either it starts po-after it ends, it starts on the same event on which it ends, or it starts po-before it ends.

If it starts po-after it ends, we can show that this prop alone is enough to cause a disallowed cycle: either the prop edge is only the eco, in which case we have a per-loc-SC cycle, or it uses cumul-fence or a terminal st to an acquire or stronger in the case of C, in either case we have an acquire or stronger on the initial thread, closing the cycle.

If it starts on the same event as it ends, this is already an hb cycle and alone is sufficient to disallow the behavior, regardless of other prop edges.

At this point, we know that all prop edges must end po-after they start. Informally, in C this is equivalent to a po edge and will inevitably be ordered by a later release or earlier acquire. Formally, if it starts po-before it ends, knowing that all prop edges must end po-after they start or a smaller cycle would disallow the execution already, if the program were single-threaded the thread position of the terminal event would be strictly greater than the thread position of the initial event, and thus it would not be a cycle. Therefore there must be

communication to another thread that doesn't use prop, and thus the thread has a release and all prop edges can be ignored in favor of release ordering.

Proof. Given an execution $E = (P, C)$, suppose there exists a cycle $(r_0, \ldots, r_n) \in \mathsf{hb}^+$ such that each r_i is a relation between events in P. If there exist at most one i such that $0 \le i \le n$, $r_i \notin (\mathsf{ppo} \cup \mathsf{st})$, i.e., r_i is the only such relation in the cycle that is not in ppo or st (and hence is only in $((\mathsf{prop} \setminus \mathsf{trivial}) \cap \mathsf{int})$), then this cycle only contains one prop edge already so set $s_j = r_j$ for all $j \in \{0,\ldots,n\}$.

Otherwise, there are at least two distinct relations $r_i \neq r_j$ such that neither is in $\mathsf{ppo} \cup \mathsf{st}$ and thus both are $((\mathsf{prop} \setminus \mathsf{trivial}) \cap \mathsf{int})$ relations. Without loss of generality, assume r_i does not start with eco. In terms of the definition of the prop relation, this means for $r_i = (r_{i0}, r_{i1}, r_{i2})$ we have $r_{i0} \in \mathsf{trivial}$. In this case, $r_i \in (\mathsf{ppo} \cup \mathsf{st})^*$, since $r_{i1} \in \mathsf{cumul\text{-}fences} \in (\mathsf{ppo} \cup \mathsf{st})^*$ and $r_{i2} \in \mathsf{st}^?$, contradicting the assumption $r_i \notin \mathsf{ppo} \cup \mathsf{st}$. Therefore each r_i must have a nontrivial $r_{i0} \in (\mathsf{overwrite} \cap \mathsf{ext})^?$.

Now on r_i there are three cases, considering that the prop relation must start and end in the same thread: it starts po-after it ends, it starts po-before it ends, or it starts and ends on the same event or relation, determined by the thread positions of the initial and terminal events in the relation.

Case 1: If the prop relation starts po-after it ends, this prop relation alone is sufficient to disallow the execution. If it is only eco, this is a per-loc-SC cycle: let s_0 be the eco relation (= prop relation) and s_1 be the po relation between the end and start of the eco relation. If it ends with a cumul-fence, let s_0 be the prop edge minus any terminal cumulative fences on the initial thread, so that it ends with eco or st. eco implies a per-loc-SC cycle, and the st provides acquire-po ordering or stronger, so we set s_1 to be the ordering guaranteed by the terminal st, whether that is an acq-po or strong-fence. If it ends with st, this is the same case as before but without needing to remove any cumulative fences.

Case 2: If the prop relation ends on the same event or relation it begins with, then this too is sufficient to disallow the execution on its own. Set s_0 to be the prop edge. This composes with itself, forming a cycle of a single prop edge.

Case 3: Since ending po-after or on the same event/relation would already disallow the execution, each prop relation must start po-before it ends. In this case, a sequence of relations comprised of prop and ppo would only serve to increase the thread position of each subsequent relation. Eventually, in order to form a cycle, there must be an st relation (since this is the only remaining relation allowed by happens-before), resulting in a release or stronger on the thread, so the prop relations can be reduced to a po-rel relation or a fence relation. Then the cycle can be reduced to ppo and st relations, using no prop edge: replace the ordering from the initial event of the first prop edge in the thread to a po-rel from that event to the release (e.g., for r_0 a prop relation $((e_0,e_1),(\ldots),(\ldots))$ and r_1 an st relation (e_n,e_{n+1}) on the same thread, the ordering simplifies to (e_0,e_n) is a po-rel and (e_n,e_{n+1}) is an st as before). □

C.1.2 C Preservation: Compound HB Implies C Coherence.

This section proves the forward direction for the equivalence of the compound and C memory models hb axioms. Theorem C.1 doesn't completely justify that a compound hb cycle on a C program is disallowed by the C hb as well. What remains is showing that, given that we have at most one prop edge (and thus at most one eco), everything else is in $(\mathsf{po} \cup \mathsf{sw})^+$, including whatever remains in prop after the eco has occurred.

Theorem C.2. Any cycle disallowed by the compound happens-before on a C program is disallowed by C's coherence axiom. This corresponds with Section 5 Theorem 1.

$$\begin{aligned}
&\forall E = (P,C) : \mathit{Execution},\\
&\quad (\forall a \in P, a.\mathit{lang} = \mathrm{C})\\
\implies &\big((\exists r_i \subset P,\\
&\qquad r = (r_0,\ldots,r_n = r_0) \in \mathsf{hb}^+)\\
\implies &(\exists s_i \subset P,\\
&\qquad s = (s_0,\ldots,s_m = s_0)\\
&\qquad \in ((\mathsf{po} \cup \mathsf{st})^+ \fatsemi \mathsf{eco}^?))\big)
\end{aligned}$$

Proof. Note that st is strictly weaker than C's release-acquire requirements: st is between two distinct threads if there are no RMWs, while C allows release-acquire on a single thread. For this proof we only need to show that C is stronger than the compound model on C programs, and for this purpose showing that compound happens-before cycles are no stronger than $((\mathsf{po} \cup \mathsf{st})^+ \fatsemi \mathsf{eco}^?)$ implies that it is no stronger than C's coherence axiom either: $((\mathsf{po} \cup \mathsf{st})^+ \fatsemi \mathsf{eco}^?)$ might allow more behavior than C's coherence axiom, but if it is disallowed by $((\mathsf{po} \cup \mathsf{st})^+ \fatsemi \mathsf{eco}^?)$ then it is definitely disallowed by C's coherence axiom.

First, consider cycles in hb^+ that don't contain a prop relation, i.e., only contain ppo and st. We show that this cycle is a subset of $((\mathsf{po} \cup \mathsf{st})^+ \fatsemi \mathsf{eco}^?)$: ppo is a subset of po, and st is a subset of st, so $(\mathsf{ppo} \cup \mathsf{st})^+ \subset (\mathsf{po} \cup \mathsf{st})^+ \subset ((\mathsf{po} \cup \mathsf{st})^+ \fatsemi \mathsf{eco}^?)$. Thus $\forall i, s_i = r_i$ suffices.

Now consider a cycle that does involve a prop relation. By Theorem C.1, we have at most one such prop relation. All other relations in the hb cycle are ppo or st and thus a subset of $(\mathsf{po} \cup \mathsf{st})^+$, so if we show prop is a subset of $\mathsf{eco}^? \fatsemi (\mathsf{po} \cup \mathsf{st})^*$, we can finish the proof after some transformations, considering prop is being used in the context of a cycle and thus we can "rotate" certain relations from the start of the cycle to the end.

prop is defined as $\{(r_0,r_1,r_2) \mid r_0 \in (\mathsf{overwrite} \cap \mathsf{ext})^? \wedge r_1 \in \mathsf{cumul\text{-}fences} \wedge r_2 \in \mathsf{st}^?\}$, so r_0 is either empty or a subset of eco. r_1 is a subset of $\mathsf{rfe} \fatsemi (\mathsf{st}^? \fatsemi \mathsf{ppo})^*$, which is a subset of $\mathsf{rfe} \fatsemi (\mathsf{po} \cup \mathsf{st})^*$, and thus is either empty or a subset of $\mathsf{rfe} \fatsemi (\mathsf{po} \cup$

$\mathsf{st})^+$. Similarly, r_2 is either empty or a subset of st. Thus, in its entirety, prop is either empty or a subset of $(\mathsf{eco}^?{}_\fatsemi(\mathsf{po}\cup\mathsf{st})^*)$.

Call the three relations used in the single prop edge allowed in the hb cycle a_0, a_1, and a_2, and the optional (and thus potentially trivial) rfe at the start of the cumulative fences $a_{1,0}$ followed by the remainder of the cumulative fences $a_{1,1}$. By extracting these three relations from prop, we now have a cycle

$$(r_0,\ldots,r_{i-1},a_0,a_{1,0},a_{1,1},a_2,r_{i+1},\ldots,r_n) \subset ((\mathsf{po}\cup\mathsf{st})^+{}_\fatsemi\mathsf{eco}^?{}_\fatsemi(\mathsf{po}\cup\mathsf{st})^+).$$

We can "rotate" this cycle to the following:

$$(a_{1,1},a_2,\ldots,r_{i+1},\ldots,r_n,r_0,\ldots,r_{i-1},a_0,a_{1,1}) \subset (\mathsf{po}\cup\mathsf{st})^+{}_\fatsemi\mathsf{eco}^?$$

which is a cycle disallowed by C's coherence axiom since it is in the format $(\mathsf{ppo}\cup\mathsf{st})^+{}_\fatsemi\mathsf{eco}^?$ now that $a_0,a_{1,0}\in\mathsf{eco}$ is at the end.

Thus all compound happens-before cycles on C programs are disallowed by the C memory model. □

C.1.3 SW–ST Equivalence. Now we need to justify the replacement of C's sw with our st, since for proofs in the other direction this simplification is too weak.

Lemma C.3. Any relation in sw that is not in st is either a po relation or implies the existence of a per-loc-SC cycle.

$$r\in\mathsf{sw}\wedge r\notin\mathsf{st}\implies r\in\mathsf{po}\vee\exists s\in(\mathsf{po}_{=\mathsf{loc}}\cup\mathsf{com})^+,s\in\mathsf{id}$$

This is useful because if the sw relation is po, it matters not if we call it sw or po, so we call it po. If it implies a cycle in per-loc-SC, this cycle alone disallows the execution in both C and the compound model, so we don't have to care about any happens-before cycles that contain such an sw relation. Thus all that remain are sw edges that are also st edges, so we replace sw with st.

Proof. The outline is as follows:

If the release is po-before the acquire, it is equivalent to a po edge. Any happens-before cycle using it remains a cycle regardless of if we call this edge sw or po, and thus we opt to call it po. If the release is po-after the acquire, then these two operations themselves form a cycle in C, one which is disallowed by the compound model's per-loc-SC as well (this also holds when the acquire and release are the same event and thus not po-before or after each other, i.e., if a RMW that is acquire and release reads from itself). Thus any release-acquire in one thread can be ignored. The same holds for release and acquire fences, as well as SC fences. Inserting RMW chains in between does not change this; all that matters is that there is some communication edge between the release and acquire that are in the same thread. □

C.1.4 C Preservation: C Coherence Implies Compound HB. This section proves the reverse direction for the equivalence of the compound and C memory models hb axioms.

Theorem C.4. Any execution disallowed by C's coherence axiom on a C program is disallowed by the compound happens-before or per-loc-SC. This corresponds with Section 5 Theorem 1.

$$\begin{aligned}&\forall E=(P,C):\mathit{Execution},\\&\quad(\forall a\in P,a.\mathit{lang}=\mathrm{C})\\\implies&((\exists r_i\subset P,\\&\qquad r=(r_0,\ldots,r_n=r_0)\\&\qquad\in((\mathsf{po}\cup\mathsf{st})^+{}_\fatsemi\mathsf{eco}^?))\\\implies&(\exists s_i\subset P,s=(s_0,\ldots,s_m=s_0),\\&\qquad(s\in\mathsf{hb}^+\\&\qquad\vee s\in(\mathsf{po}_{=\mathsf{loc}}\cup\mathsf{com})^+)))\end{aligned}$$

Proof. First, consider cycles without eco relations. These cycles are comprised entirely of po and st edges. If it were only po edges, the thread position would be strictly increasing and thus there could be no cycle: the thread position of the final event in a cycle is the same as the thread position of the initial event. Starting on thread 0, there must be an st edge that either starts or ends on this thread, and thus it starts with an acquire or stronger, or ends with a release or stronger. Using the definitions, replace po with $\mathsf{acq\text{-}po}$ or $\mathsf{po\text{-}rel}$, thus turning $(\mathsf{po}\cup\mathsf{st})^+$ into $(\mathsf{ppo}\cup\mathsf{st})^+\subset\mathsf{hb}^+$.

Now consider a cycle with an eco relation. If the cycle is only a single eco relation, then this is disallowed by per-loc-SC. If it is only po relations followed by a single eco relation, this too is disallowed by per-loc-SC. The final case is a cycle with eco and any number of po and st edges, where there is at least one st edge. We will build a prop relation, starting at this eco edge.

The eco edge is either followed by any number of po (including 0), but there eventually must be an st edge, otherwise the cycle is only eco and po which has already been covered. This st contains our first cumulative fence, and thus the start of prop's r_1. Now it continues $(\mathsf{po}\cup\mathsf{st})^+$, and these are equivalent to cumulative fences, since $\mathsf{cumul\text{-}fence}$ includes all releases or fences at least as strong as releases. Note that $\mathsf{po}{}_\fatsemi\mathsf{cumul\text{-}fence}$ does need to be simplified: for example, $(e_0){}_\fatsemi\mathsf{po}{}_\fatsemi(e_1){}_\fatsemi\mathsf{po\text{-}rel}$ is equivalent to a single $\mathsf{po\text{-}rel}$ that starts at the first event e_0 rather than at e_1. Lastly, to return to the initial thread, we have an st, potentially followed by a po to get to the initial event. If there is no po, then the cycle is complete: the prop edge ends on the same event it started on, and is thus a compound happens-before cycle. If there is a po edge, we end our prop edge before the po, so that the last relation in the prop relation is the st to the original thread, and this st is our r_2 of the prop edge. Then, since st ends in acquire or stronger, this completes the compound happens-before cycle with an $\mathsf{acq\text{-}po}$ or $\mathsf{strong\text{-}fence}$. □

C.1.5 Per-Location SC vs. C Coherence. Next, we show that cycles in the per-loc-SC axiom are disallowed by C's coherence axiom or by definition.

Theorem C.5. Any execution disallowed by the per-loc-SC axiom is disallowed by C's coherence axiom or by definition of coherence order. The final condition implies that a write is co-ordered before itself in the execution, implying that the co order is ill-formed and thus the execution is disallowed. This corresponds with Section 5 Theorem 1.

$$\begin{aligned}&\forall E=(P,C):\mathit{Execution},\\&\quad(\forall a\in P,a.\mathit{lang}=\mathsf{C})\\&\implies\big((\exists r_i\subset P,\\&\qquad r=(r_0,\ldots,r_n=r_0)\in(\mathsf{po}_{=\mathsf{loc}}\cup\mathsf{com})^+)\\&\quad\implies(\exists s_i\subset P,\\&\qquad s=(s_0,\ldots,s_m=s_0)\in((\mathsf{po}\cup\mathsf{st})^+\fatsemi\mathsf{eco}^?))\\&\qquad\vee(\exists w\in P,(w,w)\in\mathsf{co})\big)\end{aligned}$$

Proof. First, rewrite the per-loc-SC cycle by replacing each $\mathsf{po}_{=\mathsf{loc}}$ edge with its corresponding communication relation if it contains a write. If both are reads, then shorten the loop by removing all $\mathsf{po}_{=\mathsf{loc}}$ edges following the initial read that do not contain a write, then replacing the next relation in the cycle with a fr relation from the initial read to the next write. This can be done since a write that overwrites the initial read must exist: if no such write exists, the cycle would consist of only reads, implying there is no per-loc-SC cycle.

Either the resulting cycle contains a $\mathsf{po}_{=\mathsf{loc}}$ relation or it does not. Suppose it does contain a $\mathsf{po}_{=\mathsf{loc}}$ relation. Choose one such $\mathsf{po}_{=\mathsf{loc}}$ edge (r_i,r_{i+1}). The remaining relations in the cycle are communication relations, and thus in $\mathsf{eco}=\mathsf{com}^+$; thus the cycle is in $(\mathsf{po}\cup\mathsf{st})^+\fatsemi\mathsf{eco}^?$.

If no $\mathsf{po}_{=\mathsf{loc}}$ relation remains, then all reads can be removed from the cycle: every read must be part of two communication relations, one rf that ends in the read and one fr that starts with the read, as no read-to-read $\mathsf{po}_{=\mathsf{loc}}$ relations remain in the cycle. Shorten the cycle by replacing all reads r_i which are part of relations $(r_{i-1},r_i)\in\mathsf{rf}$ and $(r_i,r_{i+1})\in\mathsf{fr}$ with $(r_{i-1},r_{i+1})\in\mathsf{co}$. This implies a cycle consisting of only writes in co order, which implies for any write in that cycle it is co-before itself, violating the transitivity of the definition of co. □

C.1.6 C PSC Equivalence.

With the equivalence of the compound happens-before on single-language programs proven, we move to the propagates-before relation and its equivalence with Linux's propagates-before and C's PSCF relations. Notably, since C's SC accesses are not included in the model, we specifically compare propagates-before with C's PSCF relation and not PSCB or PSC = PSCF ∪ PSCB.

This theorem proves the reverse direction for the equivalence of the compound and C memory models SC axioms.

Theorem C.6. For all C programs such that there exists a cycle in PSCF, there exists a cycle in the compound propagates-before (pb). This corresponds with Section 5 Theorem 1.

Recall that C's happens-before (written here as hb_C) is $(\mathsf{po}\cup\mathsf{sw})^+$, and sw can be replaced with st by Lemma C.3 since any non-st sw disallows the execution regardless of other operations or cycles in the execution.

$$\begin{aligned}&\forall E=(P,C):\mathit{Execution},\\&\quad(\forall a\in P,a.\mathit{lang}=\mathsf{C})\\&\implies\big((\exists r=(r_0,\ldots,r_n=r_0)\in\\&\qquad(\mathsf{strong\text{-}fence}\fatsemi(\mathsf{hb}_C\cup(\mathsf{hb}_C\fatsemi\mathsf{eco}\fatsemi\mathsf{hb}_C)\\&\qquad\fatsemi\mathsf{strong\text{-}fence})^+))\\&\quad\implies(\exists s=(s_0,\ldots,s_m=s_0))\big)\end{aligned}$$

Proof. Firstly, we must prove that we can replace fences in C's PSCF with fence relations in the compound model. To do so, we recognize that for any fence to be part of a cycle, it must have another event po-before it and po-after it, although these events may be used as part of an sw relation rather than a po relation explicitly. A fence cannot be part of an eco relation since there can be no communication relation with a fence, so the only relation that can precede a fence is either po, implying an event po-before the fence, or st, also implying the same. We take this event, and the event that must follow the fence by the same reasoning, and create a fence relation using these events. Then, any following po or st relation can overlap on the entirety of the fence relation rather than the fence alone. For example, $(A)\fatsemi\mathsf{po}\fatsemi(\mathsf{F})\fatsemi\mathsf{po}\fatsemi(\mathsf{W})\fatsemi\mathsf{rf}\fatsemi(\mathsf{Acq})$, a sequence of events related by po followed by an sw. The fence relation relates $G=(A,\mathsf{F},\mathsf{W})$, and the po and st relations compensate by overlapping on the entire relation rather than only the fence: po is now (A,G) and st is (G,Acq).

By Lemma C.16, we can replace the $\mathsf{hb}_C\fatsemi\mathsf{eco}\fatsemi\mathsf{hb}_C$ in each r_i with $\mathsf{hb}^*\fatsemi\mathsf{prop}$.

Now we rewrite the sequence of n PSCF relations $\{r_i=(\mathsf{F}_i\fatsemi\mathsf{hb}_i^*\fatsemi\mathsf{prop}_i\fatsemi\mathsf{F}_{i+1})\}$ into a sequence of pb relations:

$$\forall i\neq n,s_i=\mathsf{prop}_i\fatsemi\mathsf{F}_{i+1}\fatsemi\mathsf{hb}_{i+1}^*\quad\text{and}\quad s_n=\mathsf{prop}_n\fatsemi\mathsf{F}_1\fatsemi\mathsf{hb}_1^*$$

Because each r_i and r_{i+1} overlap on their fence relation F_{i+1} when $i\neq n$, and when $i=n$ we know that the last PSCF relation r_n composes with r_1 via the fence F_1, the relations in each s_i compose appropriately and $s_n=s_0$. We also know that these are pb edges by definition, and that s_n composes with s_1 because hb_1^* composes with prop_1 in r_1; this is a cycle in pb^+.

Thus for any C program execution such that there exists a cycle in C's PSCF, we can construct a cycle in the compound model's pb. □

This theorem proves the forwards direction for the equivalence of the compound and C memory models SC axioms.

Theorem C.7. For all C programs such that there exists a cycle in pb, there exists a cycle in PSCF. This corresponds

with Section 5 Theorem 1.

$$\begin{aligned}&\forall E=(P,C):\mathit{Execution},\\&\quad(\forall a\in P,a.\mathit{lang}=\mathsf{C})\\&\Longrightarrow\big((\exists r=(r_0,\ldots,r_n=r_0)\in \mathsf{pb}^+)\\&\quad\Longrightarrow(\exists s=(s_0,\ldots,s_m=s_0)\in\\&\qquad([\mathsf{F}\cup\mathsf{SC}]\fatsemi(\mathsf{hb}_C\cup(\mathsf{hb}_C\fatsemi\mathsf{eco}\fatsemi\mathsf{hb}_C))\\&\qquad\fatsemi[\mathsf{F}\cup\mathsf{SC}])^+)\big)\end{aligned}$$

Proof. The construction of the PSCF cycle from a pb cycle is similar to that of the previous theorem, but in reverse.

Given a pb cycle $(r_0,r_1,\ldots,r_n)$, where each r_i is of the form $\mathsf{prop}_i\fatsemi\mathsf{strong\text{-}fence}_i\fatsemi\mathsf{hb}_i^*$, we define for all $i\neq 0$:

$$s_i=\mathsf{strong\text{-}fence}_{i-1}\fatsemi\mathsf{hb}_{i-1}^*\fatsemi\mathsf{prop}_i\fatsemi\mathsf{strong\text{-}fence}_i$$

and for $i=0$:

$$\begin{aligned}s_0&=\mathsf{strong\text{-}fence}_{n-1}\fatsemi\mathsf{hb}_{n-1}^*\fatsemi\mathsf{prop}_0\fatsemi\mathsf{strong\text{-}fence}_0\\&=\mathsf{strong\text{-}fence}_{n-1}\fatsemi\mathsf{hb}_{n-1}^*\fatsemi\mathsf{prop}_n\fatsemi\mathsf{strong\text{-}fence}_n\\&=r_n.\end{aligned}$$

prop being a subset of $\mathsf{eco}^?\fatsemi(\mathsf{ppo}\cup\mathsf{st})^*=\mathsf{eco}^?\fatsemi\mathsf{hb}_C^?$, and hb^* being a subset of $\mathsf{hb}_C^?$, this is equivalent to $\mathsf{strong\text{-}fence}\fatsemi\mathsf{hb}_C^?\fatsemi\mathsf{eco}^?\fatsemi\mathsf{hb}_C^?\fatsemi\mathsf{strong\text{-}fence}$. However, we need it in the form $\mathsf{F}\fatsemi(\mathsf{hb}_C\cup\mathsf{hb}_C\fatsemi\mathsf{eco}\fatsemi\mathsf{hb}_C)\fatsemi\mathsf{F}$. Strong-fence relations imply an event po-after the fence, so $\mathsf{strong\text{-}fence}\fatsemi\mathsf{hb}_C^?\fatsemi\mathsf{eco}^?\fatsemi\mathsf{hb}_C^?\fatsemi\mathsf{strong\text{-}fence}$ is equivalent to $\mathsf{hb}_C\fatsemi\mathsf{F}\fatsemi\mathsf{hb}_C\fatsemi\mathsf{eco}^?\fatsemi\mathsf{hb}_C\fatsemi\mathsf{F}\fatsemi\mathsf{hb}_C$. In the case that there is no eco relation, the two hb_C relations are equivalent to one hb_C relation since hb_C is closed under transitivity. If there is an eco relation, then this falls under the $(\mathsf{hb}_C\fatsemi\mathsf{eco}\fatsemi\mathsf{hb}_C)$ case. The initial and terminal hb_C relations will be part of preceding and following PSCF relations, so we truncate these from each relation r_i. Each r_i still composes with the preceding and following r_{i-1} and r_{i+1} since they overlap on the fences F_{i-1} and F_i respectively. Thus we have constructed $(s_0,\ldots,s_m)\in\mathrm{PSCF}$, a cycle disallowed by PSCF. □

C.2 Equivalence of the LKMM and the Compound Model.

C.2.1 LKMM Preservation.

The following proofs correspond with Section 5 Theorem 2.

The Linux proofs are mostly definitional.

The only "tricky" part is trivial prop vs. non-id prop.

This theorem proves the reverse direction for the equivalence of the compound and Linux kernel memory models hb axioms.

Theorem C.8. Any execution disallowed by the LKMM's happens-before axiom on a Linux program is disallowed by the compound happens-before axiom. This corresponds with Section 5 Theorem 2.

$$\begin{aligned}&\forall E=(P,C):\mathit{Execution},\\&\quad(\forall a\in P,a.\mathit{lang}=\mathsf{Linux})\\&\Longrightarrow\big((\exists r_i\subset P,\\&\qquad r=(r_0,\ldots,r_n=r_0)\in\mathrm{LKMM}\ \mathsf{hb}^+)\\&\quad\Longrightarrow\exists s_i\subset P,\\&\qquad s=(s_0,\ldots,s_m=s_0)\in\mathsf{hb}^+\big)\end{aligned}$$

Proof. LKMM ppo ⊂ compound ppo; LKMM rfe ⊂ compound st; LKMM (prop \ id) ∩ int ⊂ compound (prop \ trivial) ∩ int. □

This theorem proves the forward direction for the equivalence of the compound and Linux kernel memory models hb axioms.

Theorem C.9. Any execution disallowed by the compound happens-before axiom on a Linux program is disallowed by the Linux happens-before or Linux per-loc-SC axioms. This corresponds with Section 5 Theorem 2.

$$\begin{aligned}&\forall E=(P,C):\mathit{Execution},\\&\quad(\forall a\in P,a.\mathit{lang}=\mathsf{Linux})\\&\Longrightarrow\big((\exists r_i\subset P,\\&\qquad r=(r_0,\ldots,r_n=r_0)\in\mathsf{hb}^+)\\&\quad\Longrightarrow(\exists s_i\subset P,s=(s_0,\ldots,s_m=s_0),\\&\qquad(s\in\mathsf{hb}^+\\&\qquad\vee s\in(\mathsf{po}_{=\mathsf{loc}}\cup\mathsf{com})^+)\big)\end{aligned}$$

Proof. The compound ppo restricted to Linux is a subset of the LKMM ppo (in fact the two are equal by definition, but subset is sufficient here).

The st relation restricted to Linux ⊂ $(\mathsf{ppo}\cup\mathsf{rfe})^+$: without RMWs in between, st is either rfe, release-acquire (a subset of rfe), or fence synchronization, which is a subset of $\mathsf{ppo}\fatsemi\mathsf{rfe}\fatsemi\mathsf{ppo}$. When considering the allowed RMWs in between the start and end of st, if they are external then it is a chain of rfe; if they are internal then one of the Linux ppos captures that behavior: $(\mathsf{overwrite}\cap\mathsf{int})$ or $\mathsf{dep}\fatsemi\mathsf{rfi}$.

Compound (prop \ trivial) ∩ int: trivial is only relevant when the prop cycle is id; otherwise trivial and non-id are the same. If there is no eco start to the prop, this is ppo + st. If there is an eco edge, there are three cases:

Case 1: prop is only eco ⟹ per-loc-SC cycle, which both models would disallow.

Case 2: prop ends in cumul-fence ⟹ there exists a smaller prop edge that is not id: if eco goes directly back to the original thread, then there is a per-loc-SC cycle again. Otherwise, to get back to the original thread there must be an rfe, so stop the cycle there (i.e., set r_2 to be this rfe) and the last cumul-fence is a ppo.

Case 3: prop ends in rfe $\implies$ the eco edge is "longer": start eco at the start of the rfe (since this is still part of the communication chain), then check what case we are in again. Eventually, this is Case 2 unless the cycle is only RMWs reading each other, which is a per-loc-SC cycle (i.e., Case 1) which would already be disallowed. □

C.2.2 Linux PB Equivalence.

This theorem proves the reverse direction for the equivalence of the compound and Linux kernel memory models pb axioms.

Theorem C.10. For all Linux programs such that there exists a cycle in Linux's pb, there exists a cycle in the compound pb. This corresponds with Section 5 Theorem 2.

$$\begin{aligned}&\forall E=(P,C):\mathit{Execution},\\&\quad(\forall a\in P,a.\mathit{lang}=\mathsf{Linux})\\&\implies\big((\exists r=(r_0,\ldots,r_n=r_0)\\&\qquad\in(\mathsf{Linux\ pb})^+)\\&\qquad\implies(\exists s=(s_0,\ldots,s_m=s_0)\in\mathsf{pb}^+)\big)\end{aligned}$$

Proof. Given $(r_0,r_1,\ldots,r_n=r_0)$, set $s_i=r_i$ for all i. This is a valid compound pb cycle because Linux prop is a subset of compound prop, Linux strong-fence is a subset of compound strong-fence, and Linux hb is a subset of compound hb. In addition, the composition of relations in the LKMM is based on overlap of a single event, which is also allowed by the compound model (apply *first* and *last* "maximally"); thus any Linux pb edge is a valid compound pb edge. □

This theorem proves the forwards direction for the equivalence of the compound and Linux kernel memory models pb axioms.

Theorem C.11. For all Linux programs such that there exists a cycle in compound pb, there exists a cycle in Linux's pb. This corresponds with Section 5 Theorem 2.

$$\begin{aligned}&\forall E=(P,C):\mathit{Execution},\\&\quad(\forall a\in P,a.\mathit{lang}=\mathsf{Linux})\\&\implies\big((\exists r=(r_0,\ldots,r_n=r_0)\in\mathsf{pb}^+)\\&\qquad\implies(\exists s=(s_0,\ldots,s_m=s_0)\\&\qquad\in(\mathsf{Linux\ pb})^+)\big)\end{aligned}$$

Proof. For each prop edge r_i, we will modify this edge into a valid Linux prop edge s_i, where the only concern is that a Linux prop uses rfe instead of st in its cumulative fences. An st can be an rfe, which is valid for Linux cumulative fences, or it can contain any number of $(\mathsf{rf}\fatsemi\mathsf{rmw})$ in between the write and read. In this case, the sequence of read-modify-writes is appended to the previous cumulative fence as Linux's "rmw-sequence", and the final rf to the read is the rfe if it is external, or can be "forgotten" if it is internal since this means the last RMW is in the same thread as the next cumulative fence or the following strong-fence and thus is ordered by the fence (e.g., po-rel or mb) (Lemma C.14).

Then the strong-fence relation is the same between both Linux and the compound model (when restricted to Linux operations).

For any happens-before sequence in the compound model, it can be replaced by a sequence of ppo, rfe, and prop: ppo is identical, st is a subset of $(\mathsf{ppo}\cup\mathsf{rfe})^+$, and prop can be replaced by a Linux prop relation that starts and ends with the same event as the compound prop relation, or the compound prop relation is in id and the execution is already disallowed by both the compound model and Linux as a consequence of happens-before equivalence.

Thus any compound pb cycle implies the existence of a Linux pb cycle on that execution. □

C.3 Interchangeability

The following proof corresponds with Section 5 Theorem 3.

Theorem C.12. Interchangeability: C and Linux acquires, releases, and SC fence equivalents possess the same ordering and synchronization guarantees. This corresponds with Section 5 Theorem 3.

Proof. In the axiom definitions of hb, pb, and rb, there is no distinguishing between Linux and C releases, for example. The po-rel relation uses either Linux release or C release. Therefore, in any program that contains a cycle using a Linux release, replacing it with a C release would maintain that same cycle. If the program did not contain a cycle using a C release, replacing any C release with a Linux release would also contain no cycle using that release since if it did, replacing that Linux release with a C release would create a cycle using a C release. Therefore the existence or nonexistence of a cycle does not depend on whether a Linux release or C release is used.

The same reasoning holds for acquires and strong-fence relations. □

C.4 Supporting Lemmas

Lemma C.13. An eco edge is equivalent to an optional overwrite and rf:

$$\begin{aligned}&((e_0,e_1),\ldots,(e_{n-1},e_n))\in\mathsf{eco}\\&\quad\implies\exists i,((e_0,e_i),(e_i,e_n))\in(\mathsf{overwrite}^?\fatsemi\mathsf{rf}^?)\end{aligned}$$

Proof. First, we can simplify eco edges that contain reads that are not e_0 or e_n: $\mathsf{W}\fatsemi\mathsf{rf}\fatsemi\mathsf{R}\fatsemi\mathsf{fr}\fatsemi\mathsf{W}$ implies an overwrite from the first write to the second. If this were not the case, it would violate the guarantee that writes form a total order on each location: fr implies that the second W is coherence-order after the first since the read reads from the first write. For this reason, we can ignore any non-initial or terminal reads: a read must either start/end the eco relation, or be surrounded by two writes since there is no read-to-read relation.

Now consider what cases exist for eco edges. They can start with a read or write and end with a read or write.

1. R *to* R: For this to be in eco, there must be at least one write in between the reads that modifies the value before it is read by the second read, since there is no relation in eco that relates a read to a read directly. Choose the last such write, based on coherence order which guarantees all writes to a single location are totally ordered, as e_i. (e_0,e_i) is then fr and (e_i,e_n) is rf.
2. R *to* W: Either the write directly overwrites the read, an fr being an overwrite so choose $e_i = e_n$, or there are other writes in between so it is $\mathsf{rf}\fatsemi\mathsf{co}^+$ which is also an fr by definition; choose $e_i = e_n$.
3. W *to* R: Either an rf, choose $e_i = e_0$, or there are other writes between the W and R so choose the last such write based on coherence order as e_i.
4. W *to* W: co^+. Choose $e_i = e_n$.

□

Lemma C.14. A po followed by a po-rel or strong-fence is equivalent to a (new) po-rel or strong-fence.

$$\mathsf{po}\fatsemi\mathsf{po\text{-}rel} \subset \mathsf{po\text{-}rel}$$

$$\mathsf{po}\fatsemi\mathsf{strong\text{-}fence} \subset \mathsf{strong\text{-}fence}$$

Proof. First consider po-rel. A po-rel is defined as beginning with a po relation, and since po is transitive we take $[A]\fatsemi\mathsf{po}\fatsemi\mathsf{po}\fatsemi[B]$ to be another, "longer" po relation from A to B, i.e., $[A]\fatsemi\mathsf{po}\fatsemi[B]$.

Similarly, for a strong-fence, since it too begins with a po edge we can combine this with the prior po edge to form a new strong-fence relation. For example, $[A]\fatsemi\mathsf{po}\fatsemi[B]\fatsemi\mathsf{strong\text{-}fence}\fatsemi[C]$ becomes $[A]\fatsemi\mathsf{strong\text{-}fence}\fatsemi[C]$. □

Lemma C.15. For a C execution that contains a cycle in $\mathsf{hb}_C\fatsemi\mathsf{eco}^?$, every thread that does not only contain events in the eco relation must have a release or acquire, and thus its events are ordered by po-rel or acq-po; otherwise there is a cycle in one address (per-loc-SC).

$$\begin{aligned}
&\forall E = (P,C) : \mathit{Execution},\\
&\quad r = (r_0,r_1,\ldots r_n = r_0) \in \mathsf{hb}^+;\mathsf{eco}^?,\\
&\quad (r_s,\ldots r_t) \in \mathsf{eco}^?\\
&\quad (\forall i, i < s \vee i > t,\\
&\quad \exists j, r_i.\mathrm{thread} = r_j.\mathrm{thread}\\
&\quad \wedge (r_j.\mathrm{strength} \geq \mathrm{release} \wedge r_j.\mathrm{position} \geq r_i.\mathrm{position})\\
&\quad \vee (r_j.\mathrm{strength} \geq \mathrm{acquire} \wedge r_j.\mathrm{position} \leq r_i.\mathrm{position}))\\
&\quad \vee (\exists s = (r_{i1}, r_{i2} \ldots, r_{im} = r_{i1}) \in (\mathsf{po}_{=\mathrm{loc}} \cup \mathsf{com})^+)
\end{aligned}$$

Proof. First, consider threads in the cycle that do not contain any events that are in the eco relation. For these threads, the only relations that the events they contain can be part of are po or sw. If they are only po, there can be no cycle, since with only po edges there can be no inter-thread interaction and on the single thread the thread position is a strictly increasing monovariant: the end of the sequence of po relations will have a different thread position than the start, implying that they are not the same event.

Therefore there must be some other relation that at least one event on the thread is part of in the cycle, but by assumption this is not eco and must be sw. This implies there is an acquire or a release on the thread, but we need to ensure that this release or acquire is the po-last event or first event respectively, or if it is a fence that the corresponding write for a release fence is the last event or the read for an acquire fence is the first event.

Suppose the (program order) first event in the thread is not part of an sw relation. Then the only relation it can be in is po, but this is a contradiction: if it is only in po relations then the prior relation in the cycle must also be po, implying there is another event po-before the first event in the thread, contradicting it being po-first in the thread. Thus it must be part of an sw relation, and since it is first in the thread the previous relation must be an sw and thus the first event is an acquire (or the read for an acquire fence or stronger). A similar argument holds for the last event being a release or the write of a release fence or stronger.

Now consider a thread that does contain events that are part of the eco relation, but not only events in the eco relation. Either there is a per-loc-SC cycle or for this thread we can guarantee that at least one of the two holds: it starts with an acquire or ends with a release. Consider the arguments for why each thread that does not involve eco must start with an acquire and end with a release. Because the thread does not interact with eco, the fact that the threads cannot only be po required that it start with an sw and end with an sw. Now, because the threads that do interact with eco can use eco to interact with another thread, it could start with eco and end with eco, i.e., the first and last events in the thread can be part of an eco relation.

If it both starts and ends with eco, either there is a per-loc-SC cycle because it ends before it starts, or if it starts before it ends then the eco edge can be replaced by a po edge and the cycle simplified to one that does not use eco.

Now consider threads that contain events in eco but don't both start and end with events in eco. Then either the first or last event in the thread is not part of eco, and we first consider such threads where the first event is not part of eco. The relation that precedes this event cannot be po, or it would contradict this event being first in the thread, and cannot be eco since that too contradicts the assumption that this event is not part of eco; thus it must be st and the event implies an acquire or fence as strong as an acquire on the thread.

The same argument can be made for the last event in the thread implying a release or a fence as strong as a release on the thread. □

Lemma C.16. For a C execution, $\mathsf{strong\text{-}fence}\fatsemi(\mathsf{hb}_C \cup \mathsf{hb}_C\fatsemi\mathsf{eco}^?\fatsemi\mathsf{hb}_C)\fatsemi\mathsf{strong\text{-}fence}$ is equivalent to $\mathsf{strong\text{-}fence}\fatsemi\mathsf{hb}^*\fatsemi\mathsf{prop}\fatsemi\mathsf{strong\text{-}fence}$ or the execution is disallowed.

Proof. For a sequence of relations strong-fence⨟(hb$_C$∪hb$_C$⨟eco$^?$⨟hb$_C$)⨟strong-fence, we can first replace the sw in hb$_C$ with st via Lemma C.3. We can also replace eco$^?$ with overwrite$^?$⨟rf$^?$ via Lemma C.13.

No-eco case. First, consider the relation with no eco, i.e., strong-fence⨟hb$_C$⨟strong-fence. The hb$_C$ is some sequence of po and st, i.e., (po∪st)$^+$. As an example, consider this sequence: po⨟po⨟st⨟po. To construct a pb relation where this hb$_C$ is surrounded by strong-fences, we aim to use Lemma C.14 to compress all subsequences of po relations into po-rel and strong-fence relations, ordered by either an st that follows it (implying a release or SC fence), or by the strong-fence relation at the end of the PSCF relation. Then this example sequence, surrounded by strong-fences, would turn from strong-fence⨟po⨟po⨟st⨟po⨟strong-fence into strong-fence⨟po-rel⨟st⨟strong-fence if the st used a release, for example. This is then turned into a valid strong-fence⨟hb*⨟prop⨟strong-fence by setting hb*=po-rel⨟st and prop is empty.

In general, for a sequence

$$\text{strong-fence}⨟(\text{po}\cup\text{st})^+⨟\text{strong-fence},$$

call this S. If there are no po relations in this sequence then we are done: each st is already in the compound happens-before, so this is already a valid sequence strong-fence⨟hb*⨟strong-fence, which can be converted to the appropriate form by adding an empty prop relation.

Otherwise, consider the longest subsequence of po that includes the second-to-last relation in S. This may be empty. We construct a new sequence of relations S_1 where this subsequence is replaced by the strong-fence relation, which is allowed by Lemma C.14 which guarantees that this sequence of po, in tandem with the terminal strong-fence, is ordered by strong-fence itself. If no po relations remain, we are done. Otherwise, we can repeat this process of "compressing" subsequences of po relations preceding an st or strong-fence relation via Lemma C.14 to create a po-rel or strong-fence relation, which is part of hb.

With-eco case. Now we consider PSCF relations that contain an eco relation.

If the overwrite$^?$ and rf$^?$ are either empty or external, then this agrees with the start of prop. The eco$^?$⨟hb$_C$ can be converted to prop; however, this requires some expanding of hb$_C$ relations to ensure that we see the appropriate pattern of cumulative fences, where cumul-fence includes st$^?$⨟po-rel and st$^?$⨟strong-fence. When compressing po relations, this ensures that if there are any po edges between any two st relations, there will be an appropriate po-rel or strong-fence between those two st relations. However, if there are two st relations with no po edge between them, we must expand the relations so that we can ensure there is one. If the second st uses a strong-fence for its release synchronization, then this fence plus the preceding st is a cumulative fence, i.e., we expand st⨟st to st⨟strong-fence⨟st. The same is true for a release fence.

If it uses a release write, then we must consider what the first st uses as its acquire. If it uses a fence of any kind, this implies a po order from the read to the release, which is a po-rel for the cumulative fence, i.e., st⨟st becomes st⨟po-rel⨟st. If it uses an acquire read, then there are two possible cases: either the acquire and release are separate operations, implying a po edge between them and breaking the assumption that there was none, or they are the same operation, a RMW that is AQRL or stronger.

If the st relations overlap on a RMW, then there are two subcases: preceding the first st there is another cumulative fence (e.g., a po-rel ending at that first st), or there is not. If there is a cumulative fence preceding the first st, then this RMW is part of the (rf⨟rmw)* that can follow a cumulative fence, the (rf⨟rmw)* that follows cumul-fence as part of the following st. If there is no cumulative fence prior, then this is part of eco: the RMW reads and writes to the same location as the overwrite and rf from eco, thus we can create new overwrite and rf relations to cover this larger eco edge. Even if eco were empty, i.e., the overwrite and rf were identity relations, then rather than extending eco, the RMW (and any more RMW that follow) becomes the eco edge.

After compressing po sequences to po-rel and strong-fence relations, and expanding st⨟st to include a cumulative fence between them or become adjoined with the previous cumulative fence or eco, for a prop edge (r_0, r_1, r_2), set:

$$r_0 = \text{overwrite} \cap \text{ext}^?,$$
$$r_1 = \text{rfe}^?⨟\text{hb}_C⨟\text{strong-fence},$$
$$r_2 \text{ is empty.}$$

This is allowed because hb$_C$ has been converted to (st$^?$⨟(po-rel ∪ strong-fence))*⨟st$^?$, a valid sequence of cumulative fences plus a final st in terms of prop's r_1 and r_2.

In order to convert the initial hb$_C$ into hb*, we must apply the same po-compressing reasoning but instead of converting to po-rel and strong-fence we convert it to acq-po and strong-fence. If the initial hb$_C$ has no po relations in it, it is already a valid hb*. Otherwise, take the first po relation in hb$_C$ and the longest subsequence of hb$_C$ that consists of only po relations and contains that first po relation. This is either preceded by the initial strong-fence or by a prior st relation, so we can replace this sequence of po relations with either an acq-po relation or a strong-fence relation based on which preceded it. Similar to the previous argument with po-rel and strong-fence, we can convert the entire hb$_C$ into a valid hb* relation.

If the overwrite is not empty or external, then it is internal. Either it is also a po edge, or it is a cycle in per-loc-SC and the execution is disallowed. In this case, it is a po edge following the first hb$_C$, and hb$_C$ being transitive it can be extended to include this additional po edge. Thus any internal overwrite can be "ignored", and the new sequence of relations

is $\mathsf{hb}_C ; \mathsf{eco}^? ; \mathsf{hb}_C = \mathsf{hb}_C ; (\mathsf{overwrite}^? ; \mathsf{rf}^?) ; \mathsf{hb}_C$, where the overwrite is empty.

If rf is not empty or external, then it is internal. Either it is also a po edge, or it is a cycle in per-loc-SC and the execution is disallowed. In this case, it is a po edge preceding the second hb_C, and hb_C being transitive it can be extended to include this additional po edge. Thus any internal rf can be "ignored", and the new sequence of relations is $\mathsf{hb}_C ; \mathsf{eco}^? ; \mathsf{hb}_C = \mathsf{hb}_C ; (\mathsf{overwrite}^? ; \mathsf{rf}^?) ; \mathsf{hb}_C$, where rf is empty. □

D Additional Case Studies

D.1 ConcurrencyKit FIFO Queue (SPSC)

The FIFO Queue is a linked-list queue that maintains a persistent stub node separating the producer and consumer domains. The producer appends new entries by writing into `tail->next`, then advancing `tail`. The consumer reads `head->next` to find the next entry, then advances `head`. The data structure itself is very similar to the Michael-Scott Queue Section 6.1.1, but the underlying algorithm is much simpler and does not involve any CAS operations.

```
1 entry->data = data;
2 entry->next = NULL;
3 r0 = load_rlx(tail);
4 st_rel(r0->next, entry);
5 st_rlx(tail, entry);
```

Figure 12. *P* for CK FIFO, there are no optimizations for LKMM

```
1 r0 = load_rlx(head);
2  r1 = load_acq(r0->next); // C
3  r1 = load_rlx(r0->next); // LKMM
4 if (r1 == NULL)
5     return EMPTY;
6 read_data = r1->data;
7  st_rel(head, r1); // C
8  st_rlx(head, r1); // LKMM
```

Figure 13. C_C (blue) and C_L (red) for CK FIFO

Invariants.

1. `tail` is written only by the producer
2. `head` is written only by the consumer
3. Insertions always take place at the tail. The tail node's `next` pointer is the publication channel, and a non-NULL value signals a new entry. The new entry must be populated before being published.
4. The consumer must see all published entries before observing `head->next == NULL`.

Required Orderings. *Invariant 3* requires that a consumer reading a non-NULL value of `tail->next` must also see the fully populated entry. Therefore, the producer must ensure that the entry is populated before publishing to `tail->next`. *Invariant 4* requires that the `next` value being read is fresh, and necessarily before dereferencing the node and its contents.

P_C**: The C Producer.** Figure 12 shows the producer code for the CK FIFO. The producer simply requires a `st_rel` on `tail->next` to ensure that the data writes are ordered prior to this publication. The `next` pointer is read during linked list traversal by the consumer. The store to `tail` itself does not require a strong operation since it is local to the producer.

P_L**: The LKMM Producer.** There are no changes to the LKMM version of the producer since the C producer already uses the appropriate release semantics.

C_C**: The C Consumer.** Figure 13 shows the consumer code for the CK FIFO. The C consumer must use an explicit `ld_acq` on `head->next` to maintain *Invariant 4* and ensure that the correct entry is being read.

C_L**: The LKMM Consumer.** The LKMM consumer can use a `ld_rlx` on `head->next`, since the addr-dependency ppo and ctrl-dependency ppo will ensure that this operation takes place between the `ld_rlx(head)` and the `read_data = r1->data` operations. The store to `head` can also be relaxed since it is local to the consumer.

Evaluate Heterogenous Pairings. The `st_rel` on `next` synchronizes with the `ld_acq` on `next` in the (P,C_C) pairing, achieving release-acquire synchronization for *Invariant 4*. In the (P,C_L) pairing, the same `st_rel` satisfies `inter-start`, and the addr-dependency ppo from `head` to `head->next` on the LKMM consumer carries the ordering forward, preserving *Invariant 4*.

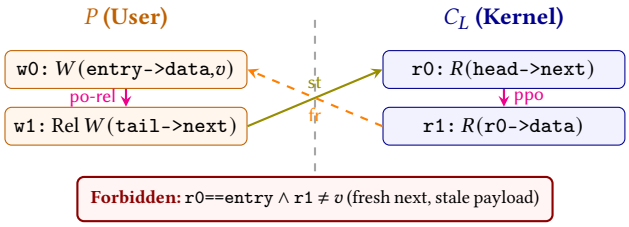

Figure 14. Heterogeneous pairing: user producer (*P*) with kernel consumer (C_L) for the CK FIFO. The st relation bridges the boundary from the user's `st_rel` to the kernel's `ld_rlx`. `w0` $\xrightarrow{\text{po-rel}}$ `w1` $\xrightarrow{\text{st}}$ `r0` $\xrightarrow{\text{ppo}}$ `r1` $\xrightarrow{\text{fr}}$ `w0` is forbidden by the compound model.

Result. All four pairings of the CK FIFO preserve the queue's invariants under the compound model. There is either a release-acquire pairing, or a release-addr-dependency-ppo that ensures the necessary ordering between the producer and consumer, with the compound model's `st` relation joining them across the boundary.

D.2 ConcurrencyKit Stack (UPMC)

The CK Stack is a classic lock-free Treiber stack, supporting a unique producer with multiple consumers. This also follows a linked-list structure, with new entries being linked to `head`, which is the top of the stack by the producer, and unlinked

by the consumer. The linking and un-linking operations are done through CAS operations on the stack's `head` pointer. The `next` pointer of each entry points to the entry below it in the stack.

Invariants.

1. The stack's `head` pointer is the shared top-of-stack pointer, written by both P and C.
2. A succesful push-CAS publishes `entry->data` and `entry->next`
3. A succesful pop-CAS must unlink the correct node, and the consumer must see the published data of the popped node.

```
entry->data = data;
r0 = ld_rlx(head);
do {
    entry->next = r0;
    r1 = cas_rel(head, r0, entry);
    if (r1 == r0)
        break;
} while (limit);
```

Figure 15. P for CK Stack, there are no optimizations for LKMM

```
r0 = load_acq(head); // C
r0 = load_rlx(head); // LKMM
while (r0 != NULL) {
    r1 = ld_rlx(r0->next);
    r2 = cas_acq(head, r0, r1); // C
    r2 = cas_rlx(head, r0, r1); // LKMM
    if (r2 == r0)
        break;
}
return r0->data;
```

Figure 16. C_C (blue) and C_L (red) for CK Stack

Required Orderings. *Invariant 2* requires that the producer must order the write to `entry->data` and `entry->next` stores before the publication push-CAS to `head`. *Invariant 3* requires that the consumer must see the published data of the popped node, which means that the load of `head` must be ordered prior to the pop-CAS, and prior to the load of `next` and `data`.

P_C**: The C Producer.** The `cas_rel` on `head` ensures that the push-CAS is ordered after the data and next stores, and also serves as the publication point for the new entry.

P_L**: The LKMM Producer.** There are no optimizations for the LKMM producer since the C producer already uses the appropriate release semantics on the push-CAS.

C_C**: The C Consumer.** The C consumer requires an explicit `ld_acq` on `head` to ensure that the read of `next` and `data` take place after the load of `head`, resulting in correctly observing the published data of the popped node. The pop-CAS uses `cas_acq` to ensure that the `head` node being unlinked is still the current `head` node, without which the unlink could lead to branches and orphaned nodes.

C_L**: The LKMM Consumer.** The LKMM consumer can relaxed the `ld_acq` on `head` to a `ld_rlx` since the addr-dependency ppo will ensure that the load of `head` takes place before the loads of `next` and `data`. The pop-CAS can also be relaxed since the consumer does not require a strong ordering between the load of `head` and the pop-CAS, as long as the correct node is being unlinked.

Evaluate Heterogenous Pairings. The stack has a single shared synchronization variable, which is `head`. The `push` operation publishes via release-CAS; pop observes via either `ld_acq` + `cas_acq` for C_C, or addr-dependency ppo and `cas_rlx` for C_L.

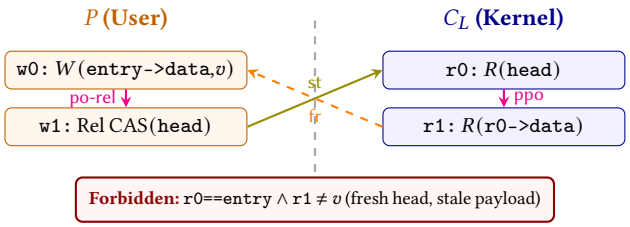

Figure 17. Heterogeneous pairing: user producer (P) with kernel consumer (C_L) for the CK Stack. The st relation bridges the boundary from the user's `cas_rel` to the kernel's `ld_rlx`. $\texttt{w0} \xrightarrow{\text{po-rel}} \texttt{w1} \xrightarrow{\text{st}} \texttt{r0} \xrightarrow{\text{ppo}} \texttt{r1} \xrightarrow{\text{fr}} \texttt{w0}$ is forbidden by the compound model.

D.3 Facebook Folly's SPSC Queue

Known simply as `ProducerConsumerQueue` in the Folly library, this is a fixed-size ring buffer supporting a single-producer and a single-consumer. There are two separate indices, the `read_idx` and `write_idx`, which are owned by the consumer and producer respectively. There is a fixed size array which is indexed into by these two indices.

```
r0 = ld_rlx(write_idx);
r1 = ld_acq(read_idx); // C
r1 = ld_rlx(read_idx); // LKMM
if (r1 != r0) {
    st_rlx(&array[r0], data);
    st_rel(write_idx, (r0 + 1) % size);
}
```

Figure 18. P for Folly SPSC Queue, there are no optimizations for LKMM

```
r0 = ld_rlx(read_idx);
r1 = ld_acq(write_idx); // C
r1 = ld_rlx(write_idx); // LKMM
if (r0 == r1)
    return EMPTY;
read_data = ld_rlx(&array[r0]);
st_rel(read_idx, (r0 + 1) % size);
```

Figure 19. C_C (blue) and C_L (red) for Folly SPSC Queue

Invariants.

1. `write_idx` is written only by the producer.
2. `read_idx` is written only by the consumer.
3. `write_idx != read_idx` indicates data is available to consume.
4. `write_idx + 1 % size != read_idx` indicates space is available to produce.

Required Orderings. *Invariant 3* requires that the producer must order data writes prior to advancing `write_idx`. Similarly, *Invariant 4* requires that the consumer must order the load of `read_idx` after reading the data.

P_C: The C Producer. Shown in Figure 18, the producer requires a `ld_acq` on `read_idx` to ensure that it does not write into a slot that the consumer is still reading. The `st_rel` on `write_idx` ensures that the data write to the array is ordered before the publication of the new `write_idx` value.

P_L: The LKMM Producer. The LKMM producer can relax the `ld_acq` on `read_idx` to a `ld_rlx` since the ctrl-dependency ppo and data-dependency ppo will ensure that the store to the array takes place after the load of `read_idx`.

C_C: The C Consumer. The C consumer requires a `ld_acq` on `write_idx` to ensure that it does not read from an empty slot. The `st_rel` on `read_idx` ensures that the load of the data from the array is ordered before the publication of the new `read_idx` value.

C_L: The LKMM Consumer. The LKMM consumer can relax the `ld_acq` on `write_idx` to a `ld_rlx` since the ctrl-dependency ppo orders the data read (and write into a buffer) after the load of `write_idx`. We retain the `st_rel` on `read_idx`.

Evaluate Heterogenous Pairings. The producer and consumer synchronize via the `write_idx` and `read_idx` variables. The C producer with an LKMM consumer still has sufficient synchronization via the `st_rel` on `write_idx` and the ctrl-dependency ppo on the consumer side. The LKMM producer with a C consumer also has sufficient synchronization via the `st_rel` on `write_idx` and the `ld_acq` on `write_idx`.

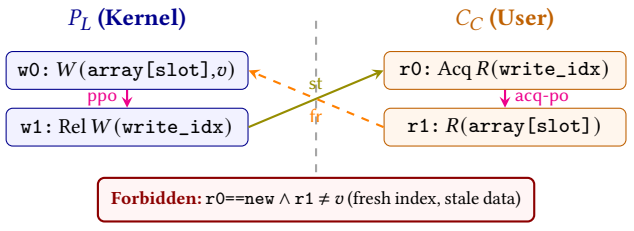

Figure 20. Heterogeneous pairing: kernel producer (P_L) with user consumer (C_C) for the Folly SPSC queue. The st relation bridges the boundary from the kernel's `st_rel` to the user's `ld_acq`. `w0` $\xrightarrow{\text{ppo}}$ `w1` $\xrightarrow{\text{st}}$ `r0` $\xrightarrow{\text{acq-po}}$ `r1` $\xrightarrow{\text{fr}}$ `w0` is forbidden by the compound model.

D.4 ConcurrencyKit SPSC Ring

The CK Ring is a single-producer single-consumer bounded ring buffer implementation found in the ConcurrencyKit library. There is a `c_head` index and a `p_tail` index, which wrap around a fixed-size array containing the entries. `c_head` is only modified by the consumer, while `p_tail` is only modified by the producer. Both producer and consumer read both indices to ascertain whether the queue is full or empty before performing their respective operations.

```
cons = load_acq(c_head); // C
cons = load_rlx(c_head); // LKMM
prod = load_rlx(p_tail);
next = prod & mask;
if (next == cons)
    return ERROR;
slot = slots[prod];
slot->data = data;
store_rel(p_tail, next);
return SUCCESS;
```

Figure 21. P_C (blue) and P_L (red) for CK Ring

```
cons = load_rlx(c_head);
prod = load_acq(p_tail); // C
prod = load_rlx(p_tail); // LKMM
if (cons == prod)
    return ERROR
slot = slots[cons];
read_data = slot->data;
next = (cons + 1) & mask;
store_rel(c_head, next);
```

Figure 22. C_C (blue) and C_L (red) for CK Ring

Invariants.

1. `c_head` must be lesser than `p_tail` as the consumer can only read payloads the producer has already written.
2. The difference between `p_tail` and `c_head` cannot exceed the buffer's capacity.
3. The producer only writes to `p_tail` and the consumer only writes to `c_head`.
4. The payload must be written before updating `p_tail`.
5. The payload must be read after reading `p_tail` to ensure visibility.
6. The payload must be read before updating `c_head` to ensure freshness for the next consumer.

Required Orderings. *Invariants 4 and 5* require that a consumer that observes the new `p_tail` index also observes the corresponding payload write.

P_C: The C Producer. Figure 21 shows the C-annotated producer. The algorithm requires `load_acq(c_head)` so the comparison with `next` uses a fresh value of `c_head`. The `st_rel` on `p_tail` ensures that the payload write to the slot is ordered before the publication of the new index.

P_L**: The LKMM Producer.** Figure 21 shows the LKMM-annotated producer. Since the control-dependency ppo will order the load prior to the payload write, `load_rlx(c_head)` can be used for the kernel-side.

C_C**: The C Consumer.** Figure 22 shows the C consumer. The `ld_acq` on `p_tail` ensures that the data read takes place after observing the new index. The `st_rel` on `c_head` ensures that the payload has been read before the producer is allowed to overwrite the slot.

C_L**: The LKMM Consumer.** The LKMM consumer can relax the `ld_acq` on `p_tail` to a `ld_rlx` since the ctrl-dependency ppo and addr-dependency ppo order the data read after the load of `p_tail`. The `st_rel` on `c_head` is retained.

Evaluate Heterogenous Pairings. The producer and consumer synchronize via the `p_tail` and `c_head` variables. The LKMM producer with a C consumer has sufficient synchronization via the `st_rel` on `p_tail` and the `ld_acq` on `p_tail`. The C producer with an LKMM consumer also has sufficient synchronization via the `st_rel` on `p_tail` and the ctrl-dependency ppo on the consumer side.

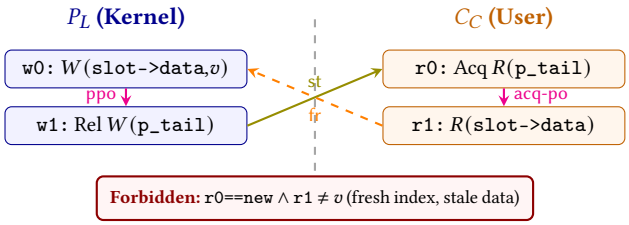

Figure 23. Heterogeneous pairing: kernel producer (P_L) with user consumer (C_C) for the CK Ring. The st relation bridges the boundary from the kernel's `st_rel` to the user's `ld_acq`. $\texttt{w0} \xrightarrow{\text{ppo}} \texttt{w1} \xrightarrow{\text{st}} \texttt{r0} \xrightarrow{\text{acq-po}} \texttt{r1} \xrightarrow{\text{fr}} \texttt{w0}$ is forbidden by the compound model.

Result. All four pairings of the CK Ring preserve the ring buffer's invariants under the compound model. The `st_rel` on `p_tail` pairs with either the C consumer's `ld_acq` or the LKMM consumer's ctrl/addr-dependency ppo, with the compound model's `st` relation joining them across the boundary.